Estimating How Much Memory a Program Uses over Time from Partial Instrumentation of Its Memory Accesses

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

Knowing how much memory a program touches, and when, helps compilers, runtime systems and job schedulers decide where to place data and how many programs fit on one machine. The most direct way to measure it is to record every memory access with a binary instrumentation tool such as Pin, but recording every access slows a program down by two to three orders of magnitude. MemPrint reduces that cost by recording only a random fraction of the accesses and by learning, from small inputs that were recorded completely, how to scale the sampled result up to the true total. MemPrint predicts a single number for a whole run. In this report we extend the problem to the number of bytes in use at every point of a run, with freed memory removed, and we describe every approach we tried, including those that failed, because the failures explain the design that works. We find four things. First, MemPrint's definition of the footprint, which adds up the largest access at each start address, counts overlapping accesses more than once, by up to 3.4× on a database; we therefore count the bytes touched, and we stop counting the memory allocator's own bookkeeping. Second, estimators that work from a random sample of memory accesses cannot tell memory that is used for the first time from memory that is used again, and at a sampling rate of 1 in 25 accesses the best such estimator still has 24% mean error on a larger input. Third, sampling memory words instead of accesses removes that difficulty, because every word is chosen with the same probability no matter how often it is used; this estimator needs no training, and its mean error is 1.6% to 4.9% on PolyBench at 1 in 100 words or denser and 0.3% to 6.8% on miniVite with 1, 4 and 16 threads. Fourth, sampling words still requires a check on every access. We therefore check accesses only during short windows of time, and between windows we track only memory allocations, the pages that the operating system reports as touched, and, for memory that the allocator reuses, the pages written since each allocation. Watching 5% of the run, the estimate of the peak is within 0.1% on PolyBench, within 1.8% on miniVite, and within −3.7% to +6.7% on graph kernels, a neural network, three language interpreters, a database and a compiler; the true peaks are consistent with Valgrind's heap profilers and never exceed the operating system's resident memory. The estimate costs about 3.5× native time on PolyBench and 15× to 18× on miniVite at 5% watched, and 2.2× to 2.7× and 13× to 17× at 1% watched, which is as accurate. With a correction for partly used pages, 1% watched is within −4.9% to +5.9% at the peak on every program; with 8 MPI processes it is within 2.1% of every process's peak, and under the jemalloc and tcmalloc allocators within −7.0% to +2.8%.

Contents

1 Introduction **4**

2 Terms **5**

3 Background **6**
3.1 Dynamic binary instrumentation with Pin . . . 6
3.2 MemPrint . . . 7

4 Measuring the live footprint over time **8**
4.1 Removing freed memory . . . 8
4.2 The allocator's own memory . . . 9
4.3 Counting bytes touched . . . 9
4.4 Snapshots and partial recordings . . . 9
4.5 Ordering events across threads . . . 10
4.6 Tests . . . 10

5 Evaluation method **10**
5.1 Workloads, inputs and machine . . . 10
5.2 Ground truth and error metrics . . . 11
5.3 Validation of the ground truth . . . 12

6 Estimating from sampled references **13**
6.1 The α model at every snapshot . . . 13
6.2 Estimators of unseen species . . . 14
6.3 An estimator that uses the known sampling rate . . . 15
6.4 A hybrid of the known-rate estimate and a learned correction . . . 15

7 Sampling words **17**
7.1 Method . . . 17
7.2 Results . . . 18

8 Windowed sampling **19**
8.1 Counting time between windows . . . 19
8.2 Switching between watching and not watching . . . 20
8.3 Tracking live blocks . . . 21
8.4 Page residency . . . 21
8.5 Written pages of reused blocks . . . 22
8.6 Word sampling inside windows . . . 22
8.7 The estimate . . . 23
8.8 Windows opened by large allocations . . . 23
8.9 Approaches that did not work . . . 23

9 Evaluation of windowed sampling **24**
9.1 PolyBench . . . 24
9.2 miniVite . . . 25
9.3 Heap churn . . . 26
9.4 Programs that reuse their heap . . . 26
9.5 Cost . . . 27
9.6 Fewer windows, cheaper switching and partial pages . . . 28
9.7 Many MPI processes . . . 30
9.8 Other allocators . . . 30

10 Overhead of every method **31**

11 Discussion and limitations **33**

12 Related work **35**

13 Next steps **36**

14 Conclusion **37**

A Options of the Pin tool **41**

B Output files **41**

1 Introduction

A program's *memory footprint* is the amount of memory it actually uses. Many decisions depend on it. A job scheduler needs to know how much memory a job will need before it places the job on a machine. A system with two kinds of memory, for example fast DRAM and slower but larger memory, needs to know which data is used and when, so that it can keep that data in the fast memory [1, 39]. A compiler that reorganizes data needs to know which parts of an array a loop touches.

Measuring the footprint is harder than it seems. A heap profiler such as Valgrind's Massif [41] or DHAT [32] records every call to the memory allocator, but an allocation only says that memory was reserved, not that it was used, and a program can reserve far more memory than it touches: in our measurements, a neural network allocates twice what it touches and a heap-churn test four times. A heap profiler also does not see the stack, global variables, or memory that libraries request directly from the operating system. The most direct measurement records every memory access with a dynamic binary instrumentation tool, which is a tool that inserts extra code into a running program. Pin [28], Valgrind [33] and DynamoRIO [7] are such tools. Recording every access slows a program down by two to three orders of magnitude, which makes the measurement impractical for large programs.

MemPrint [31] reduces that cost by recording only a random fraction of the memory accesses. It then learns, from small inputs of the same program that were recorded completely, a correction factor that scales the footprint seen in the sample up to the true footprint. MemPrint predicts one number per run: the total amount of memory touched over the whole run.

Many uses need more than one number. The amount of memory in use changes during a run, because a program allocates memory, uses it and frees it again. The largest amount in use at any one time, which we call the *peak*, decides how much memory a machine must have, and the time at which the peak occurs decides when that memory is needed. In this report we extend MemPrint to estimate the number of bytes in use at every point of a run, with freed memory removed. We record every approach we tried, including the approaches that failed, because the reasons for the failures explain the design that works.

In this work, we make the following contributions:

- We extend the MemPrint Pin tool so that it removes freed memory from its measurements, writes the measurements at regular points in time, and can stop recording partway through a run. We find and fix two errors in how the tool ordered events from different threads, which made the measured peak of a multithreaded test up to 19% too low, and two errors that kept the allocator's own memory accesses in the measurement (Section 4).

- We show that MemPrint's definition of the footprint counts overlapping accesses more than once, by 1.4× to 3.4× on real programs, and we measure the bytes touched instead (Section 2 and Section 4.3).

- We show that estimators which work from a random sample of memory accesses cannot reach low error at sparse sampling rates, because such a sample cannot tell new memory from memory that is used again. This holds for the MemPrint model applied at every point in time, for two estimators from ecology, and for a new estimator that uses the known sampling rate together with a learned correction (Section 6).

- We propose sampling memory words instead of memory accesses, and show that it estimates the memory in use over time without any training (Section 7).

- We propose windowed sampling, which samples memory words only during short windows of time, and which tracks allocations, touched pages and written pages for the whole run at low cost (Section 8 and Section 9).
- We evaluate every estimator against complete recordings on PolyBench [38], miniVite [18], the GAP graph kernels [3], the darknet neural network framework [40], the Python, Lua and Perl interpreters, the SQLite database [43], the GCC compiler and a synthetic test that reuses freed memory, we validate the true peaks against Valgrind's heap profilers and the operating system, and we list the limitations and the next steps (Section 11 and Section 13).
- We reduce the cost of windowed sampling further, with a mode that opens no windows, a correction for partly used pages and cheaper switching between watching and not watching, and we extend it to MPI programs with up to 8 processes and to the jemalloc and tcmalloc allocators (Section 9.6, Section 9.7 and Section 9.8).

The rest of this report is organized as follows. Section 2 defines the terms we use. Section 3 describes Pin and MemPrint. Section 4 describes the changes we made to the Pin tool to measure memory over time, and Section 5 describes how we evaluate an estimate. Sections 6 to 9 present the three families of estimators in the order in which we developed them. Section 11 discusses limitations, Section 12 discusses related work, and Section 13 lists next steps. The appendices list the tool's options and output files.

2 Terms

We define here the terms used in the rest of the report, in an order in which each definition uses only earlier ones.

Memory reference. A *memory reference* is one read or one write of memory by one machine instruction. An instruction that both reads and writes the same memory location, for example an instruction that adds a register to a value in memory, makes two references. Each reference has a *start address*, which is the address of the first byte it touches, and a *size*, which is the number of bytes it touches, for example 8 bytes for a 64-bit integer.

Footprint. The *footprint* of a set of memory references is the number of distinct bytes they touch, that is, the size of the union of the ranges $[a, a + \text{size})$ over all references. For example, an 8-byte read at address A and an 8-byte read at $A + 4$ add 12 bytes, because together they cover 12 bytes.

MemPrint defines the footprint differently, as the sum, over all distinct start addresses, of the largest size seen at each start address. Under that definition the same two reads add 16 bytes, because they have different start addresses. That definition therefore counts overlapping accesses more than once, and code that reads data at many byte offsets, for example a record parser, a string function or an unaligned copy, can make it several times larger than the memory touched: an 8-byte read at every byte of a 1 MB buffer gives 8.5 MB under it, against 1.15 MB of bytes touched, including the runtime's own memory. We call it the *start-address footprint*. The tool still computes it when free tracking and windows are off, so that MemPrint's outputs do not change, and Section 4.3 explains why we changed the definition.

Live footprint and peak. The *live footprint* at a point in time t is the footprint of all references made up to t, after removing every byte inside a memory range that the program freed before t.

When the program frees a range, its bytes stop counting at that moment, and if the program later uses them again, they count again. The *peak* is the largest live footprint over the whole run.

Time. We measure time in memory references executed since the start of the run, rather than in seconds. Instrumentation slows different parts of a program down by different factors, so seconds measured under instrumentation do not correspond to seconds in a run without instrumentation, whereas the number of references executed is the same.

Snapshot and timeline. A *snapshot* is a record of the current measurements, which the tool writes every N references. The sequence of snapshots of one run is its *timeline*, and the live footprint over the timeline is the run's *footprint curve*.

Sample and sampling interval. A *sample* is a subset of the references (or of the memory words) of a run, chosen at random. A sample with *sampling interval* k, or sampling rate $1/k$, contains each element with probability $1/k$. We write "1 in k" for such a sample.

Allocation, block and page. An *allocation* is a request for memory, made to the C library's memory allocator, for example by calling `malloc`, or to the operating system, by calling `mmap`. A *block* is the memory range that one allocation returns. The operating system manages memory in *pages*, which are 4 KB on our machine, and a block can start and end inside a page. A page is *resident* when the operating system has assigned physical memory to it. For memory that is not backed by a file, which the operating system calls anonymous memory, this happens at the first read or write of the page.

Native time and overhead. The *native time* of a run is its running time without any instrumentation. The *overhead* of a tool is the running time with the tool divided by the native time, written for example as $3\times$.

Ground truth. The *ground truth*, or truth, of an input is the footprint curve measured from a complete recording of every reference of a run on that input.

3 Background

3.1 Dynamic binary instrumentation with Pin

Pin [28, 24] runs a program in the same process as a *Pin tool*, which is a library that tells Pin what to observe. Pin does not run the program's machine code directly. Instead, just before a piece of code runs for the first time, Pin translates it into a new version that also calls the tool, stores the new version in a memory area called the *code cache*, and runs the new version from there. The unit that Pin translates is a *trace*, which is a straight sequence of instructions that is entered only at its first instruction and may be left at several points. A trace consists of *basic blocks*, each of which is entered only at its first instruction and left only at its last.

A Pin tool has two kinds of functions. An *instrumentation routine* runs once, when Pin translates a piece of code, and decides which calls to insert into the translation. An *analysis routine* is a function that the inserted calls invoke every time the translated code runs. For example, an instrumentation routine can insert, before every instruction that reads memory, a call to an analysis

routine that receives the address read. Calling an analysis routine on every memory reference is what makes complete recording slow.

Pin provides several ways to make analysis cheaper. It can copy a short, simple analysis routine directly into the translated code instead of calling it, which is called *inlining*. It provides a pair of calls, which we call a *test and action pair*, in which a short test routine is inlined and a longer action routine runs only when the test returns true. It provides *trace buffers*, for which the tool describes a record layout, Pin writes one record per instrumented instruction into a buffer that belongs to the running thread, and Pin calls the tool when a buffer is full. Finally, a tool can discard all translations in the code cache with the function `PIN_RemoveInstrumentation`, after which Pin translates code again the next time it runs, and it can make a thread continue from a given machine state with `PIN_ExecuteAt`.

3.2 MemPrint

The MemPrint Pin tool has two modes.

Splitter. The *splitter* records every memory reference. It writes each reference (the instruction's address, the start address, the size, and whether it is a read or a write) into a trace buffer of 1024 pages per thread, and when a buffer is full it processes the references in order. It keeps the exact start-address footprint in a hash table from start address to largest size. In addition, for each sampling interval k in a list (by default 100, 250, 500, 750, 1000, 2500, 5000, 7500, 10000, 25000, 50000, 75000 and 100000), it keeps 20 *bins*, each of which is a footprint of its own. A reference enters an interval's bins with probability $20/k$, decided by a linear congruential random number generator (a generator that computes $x \leftarrow 1664525\,x + 1013904223$ modulo 2^{32}), and then goes to one of the 20 bins chosen uniformly at random. Each bin therefore holds an independent 1-in-k sample of the references. The splitter's outputs are the training data of MemPrint.

Sampler. The *sampler* records only 1 in i references. It decides with the same random number generator, in an inlined test, whether a reference is sampled, so that a reference that is not sampled costs only the test. It places each sampled reference into K distinct bins out of s, where K is drawn from a Poisson distribution with mean $\lambda = s/r$, using Knuth's method of multiplying uniform random numbers until the product falls below $e^{-\lambda}$. Drawing a Poisson number of copies of each element is a way of building bootstrap samples [14] from data that arrives one element at a time [20]. A bin then holds a sample with an effective interval of $i\,s/\mathrm{E}[\min(K, s)]$, which is about $i\,r$ when λ is much smaller than s.

The α model. MemPrint defines $\alpha = F/F_{\mathrm{bin}}$, where F is the true footprint and F_{bin} is the footprint of one bin, and it models $\log \alpha$ as a linear function of seven features of the bins. The features are $\log \sigma$, $\log k$ and $\log m$, where σ is the standard deviation of the footprint across the bins of an interval, k is the sampling interval and m is the bin footprint, together with the products $\log m \log \sigma$, $\log k \log \sigma$, $\log m \log k$ and $\log m \log \sigma \log k$. The model is fitted by least squares on the splitter's bins of the small inputs of a program, using one of three subsets of the intervals: all of them (subset NZ), the intervals whose spread grows steadily with the input size (subset MT), or the best such interval together with one smaller and three larger intervals (subset L2O). The model is then applied to the bins of a sampler run on a large input.

Reproduction. We reorganized the original scripts into one Python package and reproduced all 16 data figures of the MemPrint manuscript pixel for pixel, together with its tables. The only

difference is the average row of the manuscript's error table, which was computed from an earlier version of the data.

4 Measuring the live footprint over time

We added several options to the Pin tool, which we call `memprint_trace`. With free tracking, snapshots and windows turned off, the tool's output files are byte for byte identical to those of the original tool, which we checked on PolyBench with fixed random seeds and with address space layout randomization turned off. Appendix A lists all options.

4.1 Removing freed memory

With `-track_frees 1`, the tool removes freed memory ranges from every footprint it keeps. To find the ranges, it inserts calls at the entry and at the exit of the C library's allocation functions: `malloc`, `calloc`, `realloc`, `memalign`, `aligned_alloc`, `valloc`, `posix_memalign` and `free`. At the entry of an allocation function it saves the requested size and, for `realloc`, the old address. At the exit it records the new block's address and size in a table. When a block is freed, the tool looks up its size and removes its range from the footprints. For `realloc`, the tool removes the old block if the block moved, or the part that was cut off if the block shrank in place. Several details were necessary to make this correct.

First, the GNU C library's `free` ends with a jump to another function rather than with a return instruction, so Pin never reports that it has returned. The tool therefore removes a block at the entry of `free`.

Second, allocation functions call each other, for example `calloc` may call `malloc`, so one call from the program can produce several entry events. The tool counts how deeply allocation calls are nested in each thread and records only the outermost call. A call is nested only if it comes from inside an instrumented allocation function and runs with a lower stack pointer than the outermost call. Any other entry starts a new outermost call, because the previous one may have left without its exit being seen: the first time the allocator is used, the GNU C library's `calloc` calls `malloc` through an initialization function and then jumps to `memset` instead of returning. An earlier version that relied on the stack pointer alone treated every later allocation of darknet as nested in that first `calloc` and recorded 53 KB of its 513 MB.

Third, a function can be found more than once in the program's libraries. Calls to library functions go through small stubs in a section named `.plt`, which jump to the real function and never return, and some functions have several names at the same address, for example `memalign` and `aligned_alloc`. The tool skips stubs in `.plt` sections and instruments each function address only once.

Fourth, memory can also be returned to the operating system with the `munmap` system call, which some libraries call directly. The tool therefore watches every `munmap` system call, at its entry to learn the range and at its exit to learn whether it succeeded, and then removes the whole pages in the range.

To remove a range quickly, each footprint keeps, in addition to its table of addresses, an index from page number to the addresses recorded in that page, and, for the footprint of bytes touched, one bit per byte of every touched page. Removing a range then visits only the pages in the range, or only the recorded pages if there are fewer of them.

Ordering removals within a thread. In the splitter and the sampler, references wait in a thread's trace buffer until the buffer is full. A free must therefore not be applied to the footprints

before the references that the thread made before the free. When a thread frees a block, the tool records the release together with the current fill position of the thread's buffer, and it applies the release when it has processed the buffer up to that position.

4.2 The allocator's own memory

The allocator keeps bookkeeping data next to the blocks it hands out. The GNU C allocator writes a size field just below every block, and when a block is freed it writes the links of its free lists into the freed memory. These writes are memory references like any other, so the tool used to count them. They land outside every live block, and nothing ever frees them, so they accumulated. A test that allocates, writes and frees 200,000 blocks of 32 to 64 bytes three times ended with 5.7 MB "live" besides its 1.6 MB array of pointers, and its peak read 23.4 MB against about 14.4 MB of data. The Lua interpreter ended a run with 16.7 MB live and 0.1 MB allocated. The bookkeeping made up 15% of Lua's peak, 7% of Perl's and 16% of SQLite's.

With free tracking or windows on, the tool therefore finds the allocator's own code when the C library is loaded, by the names of its functions in the library's symbol table: the public functions and the internal ones such as `_int_malloc` and `_int_free`. The allocator is one source file, so its code is contiguous, and the tool takes the address range that these functions span. A memory reference made by an instruction in that range still counts as time but goes into no footprint. Copies and fills that the allocator delegates to `memcpy` and `memset`, for example the zeroing in `calloc` and the copy in `realloc`, still count, because they write the program's block. With this change the test above peaks at 14.45 MB and ends at 1.7 MB.

A second, smaller leak appeared when we counted bytes touched (Section 4.3). String and copy functions read a few bytes past the end of a block, inside the slack that the allocator adds to round the block up, and those bytes were never freed, because a free removed only the requested size: Lua ended with 5.3 MB live and nothing allocated. A freed block now releases its whole chunk, which is the size field below it, the block and its slack, read from the size field at the entry of `free` and checked against the requested size. If the field does not match a chunk of that size, as for a block that the allocator mapped directly, the tool releases only the requested size; with another allocator it releases the usable size that the allocator reports (Section 9.8). Lua now ends at 0.3 MB.

4.3 Counting bytes touched

With free tracking or windows on, the tool counts the bytes touched (option `-footprint bytes`). Each footprint keeps one bit per byte of every page it has touched, adds the bits that a reference sets, and clears the bits of a released range. It still keeps, for each start address, how often it was sampled, because the estimators of Section 6 need those counts.

We changed the definition because the start-address footprint measured the access pattern as much as the memory. On the same runs, the start-address peak was 3.4× the bytes touched for SQLite (263 MB against 76 MB, for a process whose resident memory peaks at 83 MB), 1.5× for darknet and 1.4× for the GAP kernels, and 1.0× for the PolyBench kernels, which read aligned 8-byte numbers. Windowed sampling (Section 8) also needed a correction, which we called the density, to convert the bytes on touched pages into a start-address footprint, and that correction was its largest source of error (Section 8.6).

4.4 Snapshots and partial recordings

With `-snapshot N`, the tool appends the current state of every footprint it keeps to a timeline file every N references (Appendix B). Each row holds the time, the sampling interval, the bin number,

the footprint in bytes, the number of distinct start addresses, the number of references counted, the number of bytes freed so far, and five counts used in Section 6. The C library that Pin provides to tools took about 7 milliseconds to format one snapshot with its standard formatting functions, which slowed down runs with frequent snapshots, so the tool converts numbers to text itself.

With `-stop N`, the tool writes all its outputs after N references and detaches from the program with `PIN_Detach`, so that the program finishes without instrumentation. This option produces partial recordings.

4.5 Ordering events across threads

Because each thread has its own trace buffer, the tool processes the references of different threads in the order in which their buffers fill, not in the order in which the references happened. A free in one thread could therefore be applied before another thread's earlier references to the same block, and those references then added the freed block back to the footprint. We found this error with a test in which 8 threads each allocate and touch a block, wait for each other with 12 MB in use, and then free their blocks. With buffers of 1024 pages, the complete recording reported a peak of only 9.8 to 10.8 MB. The tool now uses buffers of 16 pages when snapshots or free tracking are on, which limits how far the threads can drift apart, and the same test then gives 12.08 MB on every run. Without those options the tool keeps the original 1024-page buffers, so its default outputs do not change.

4.6 Tests

A test suite of small C and C++ programs with known footprints checks the tool. The programs allocate, touch and free blocks with `malloc` and `free`, grow and shrink blocks with `realloc` both in place and by moving them, use `calloc` (also as the program's first allocation) and C++ `new`, map and partly unmap memory with `mmap` and `munmap`, allocate in one thread and free in another, use many small blocks, touch the same memory repeatedly, run 8 threads at once, and reuse freed heap memory. For each program, the suite checks the peak of the timeline, the number of bytes freed, the footprint at exit and the number of snapshots, and it also checks the `-stop` option.

5 Evaluation method

5.1 Workloads, inputs and machine

PolyBench. PolyBench/C 4.2.1 [38] is a collection of small numerical kernels. We use four of them: 2mm (two matrix multiplications), gemm (one general matrix multiplication), jacobi-2d (a two-dimensional stencil, which updates each point of a grid from its neighbors, repeated for many time steps) and atax (a matrix and vector product). As in MemPrint, we compile them without optimization. We added three sizes between the standard ones, so that the sizes MINI, MINI2, MINI3, SMALL, SMALL2, SMALL3 and MEDIUM are available for the studies of Sections 6 and 7, and we use the standard LARGE size, whose live peaks are 27 to 37 MB and whose runs make 28 to 49 billion references, for windowed sampling. PolyBench allocates its arrays at the start of a run and frees them at the end.

miniVite. miniVite [18] is a small application that finds communities in a graph with the Louvain method [6], which repeatedly moves each vertex to the neighboring community that most improves a quality score. It generates a random geometric graph, which connects points that lie close to each

other in the unit square. It uses MPI for communication between processes and OpenMP [35] for threads within a process. We run one MPI process with 1, 4 or 16 OpenMP threads, on graphs of 1024 to 65536 vertices, and we set `OMP_WAIT_POLICY=passive`, so that idle threads sleep instead of repeatedly checking for work. miniVite allocates and frees memory throughout its run: at 65536 vertices it makes about 8 million calls to `malloc` and `free`.

Building miniVite. We first built miniVite against the OpenMPI [17] library of the operating system. That build touched about 175 MB at every graph size, mostly in buffers that the MPI library sets up at the start, and its threads, which repeatedly checked for work while waiting, made 97% of its 1.76 billion references at 1024 vertices. We rebuilt it against OpenMPI 5.0.5 from the Spack package manager, the version used for the MemPrint manuscript. With that build and with sleeping idle threads, a complete recording at 1024 vertices gives a total start-address footprint of 6.8 to 7.0 MB, which matches the 6.84 MB in the manuscript, from 44 million references.

Programs that reuse their heap. To test windowed sampling on real programs, we use the GAP benchmark suite [3] (breadth-first search, PageRank, connected components and single-source shortest paths on a synthetic graph with 2^{18} vertices), darknet [40] classifying four images with AlexNet [27] and random weights, and six programs that allocate and free memory constantly: CPython 3.6 serializing, parsing and sorting records, Lua 5.3 [23] growing tables and concatenating strings, Perl 5.26 building hashes and strings, SQLite 3.26 [43] building, indexing, joining and vacuuming an in-memory database of 300,000 rows, and GCC 8's compiler proper (`cc1`) compiling the SQLite source file at `-O0`. They run for 1 to 4 s natively and make 1.2 to 10.8 billion references. All but darknet, GAP and Python keep most of their memory in reused heap blocks (Section 8.3).

For runs with several MPI processes we use LULESH 2.0, a proxy application for shock hydrodynamics on an unstructured mesh, built with MPI at `-O3`, with 8 processes, and miniVite with 4 processes (Section 9.7). To test allocators other than the GNU C library's, we run Lua and SQLite with jemalloc 5.3.0 and with tcmalloc from gperftools 2.18, loaded with `LD_PRELOAD` (Section 9.8).

Machine. All runs use Pin 3.30 on a machine with two Intel Xeon E5-2620 v4 processors (16 cores and 32 hardware threads in total) and 62 GB of memory, running Rocky Linux 8.10 with Linux 4.18, GCC 8.5 and the GNU C library 2.28.

5.2 Ground truth and error metrics

We obtain the ground truth of an input from a splitter run with `-track_frees 1`, with the same number of threads as the run being evaluated.

We compare an estimated curve $\hat{F}(t)$ with the true curve $F(t)$ using three numbers, each expressed in percent. Because two runs of a multithreaded program do not execute exactly the same number of references, we compare times as fractions of each run, so that the estimate at 40% of its run is compared with the truth at 40% of its run.

- The *mean error* is the mean, over the snapshots of the estimated run, of $|\hat{F}(t) - F(t)|/F(t)$. It is also known as the mean absolute percentage error.
- The *error of the peak* is $(\max_t \hat{F}(t) - \max_t F(t))/\max_t F(t)$. It compares the largest estimate with the largest true value, wherever each occurs.
- The *error at the peak* is $(\hat{F}(t^*) - F(t^*))/F(t^*)$, where t^* is the time at which the true footprint peaks.

Table 1: True peak (bytes touched, complete recording), the live allocations that the tool tracks, and three independent measurements, in MB. Single thread.

Workload	True peak	Tracked allocations	Massif heap	DHAT	Massif pages	Resident
2mm LARGE	37.1	37.1	37.0	37.0	43.5	38.3
gemm LARGE	29.1	29.0	29.0	29.0	35.4	30.1
jacobi-2d LARGE	27.1	27.1	27.0	27.0	33.5	28.3
miniVite 65536	28.2	52.0	43.2	43.2	206.2	47.5
miniVite 32768	15.4	30.7	23.5	23.5	206.2	35.6
GAP bfs	72.8	72.5	72.4	72.4	95.5	76.1
GAP sssp	133.8	135.5	135.4	135.4	158.5	139.2
darknet	269.9	531.5	531.5	532.2	548.5	285.7
Python	95.7	98.5	94.7	95.1	383.3	114.2
Lua	55.5	52.4	52.4	52.4	86.4	65.7
Perl	54.9	53.6	54.7	54.7	304.2	64.6
SQLite	76.2	75.6	78.4	78.4	102.6	82.5
GCC `cc1`	105.4	113.9	16.3	16.3	384.8	134.3
heap churn	1.3	4.8	4.9	4.9	11.6	6.2

Training and testing. Estimators that need training are trained, as in MemPrint, on the smaller inputs of a program and tested on an input that was not used for training. We test on the largest input, which we call *extrapolation*, and on the middle input, which we call *interpolation*, each time training on all the other inputs.

5.3 Validation of the ground truth

We checked the true peaks against three independent measurements: the program's peak resident memory reported by the operating system (`/usr/bin/time`), Valgrind's Massif heap profiler [41], both of the heap and with `-pages-as-heap`, which counts every mapped page, and Valgrind's DHAT [32], which reports the heap at its global maximum. Table 1 lists them.

The true peak never exceeds the resident memory (it is 21% to 97% of it), as it should not, because resident memory counts whole pages, code and libraries, and memory that was freed but not returned to the operating system. Where a program touches everything it allocates, as PolyBench and GAP do, the true peak is 95% to 97% of the resident memory. The live allocations that the tool tracks agree with Massif and DHAT to within 0.1 MB on PolyBench, GAP, darknet and Lua, and to within 1 to 3 MB on Perl and SQLite, where Massif reports the exact peak whereas the tool reports the largest snapshot. They are larger where a program maps memory itself, which Massif's heap mode does not count: GCC's garbage collector (98 MB), miniVite's MPI library (7 to 9 MB) and Python's small-object allocator (3.8 MB). Allocated is not touched: darknet allocates 532 MB and touches 270 MB, miniVite touches about half of what it allocates, and the churn test a quarter, so heap profilers overstate the footprint of such programs by 1.5× to 4×. DHAT also counts accesses per byte offset, but only for blocks of up to 1 KB, which cover too little of these heaps to check the bytes touched directly.

Figure 1 puts these peaks side by side, as ratios to the true peak, together with the windowed estimate of Section 8 at 5% watched. Massif's heap peak and DHAT's heap at its global maximum agree within 0.4% on every workload. The windowed estimate is within −4% to +6% of the true peak everywhere except the churn test (+23%, from whole pages), whereas the heap profilers agree with the truth only where a program touches what it allocates: they are 1.5× to 3.8× too high on miniVite, darknet and the churn test, and 0.15× on GCC, whose garbage collector maps its own

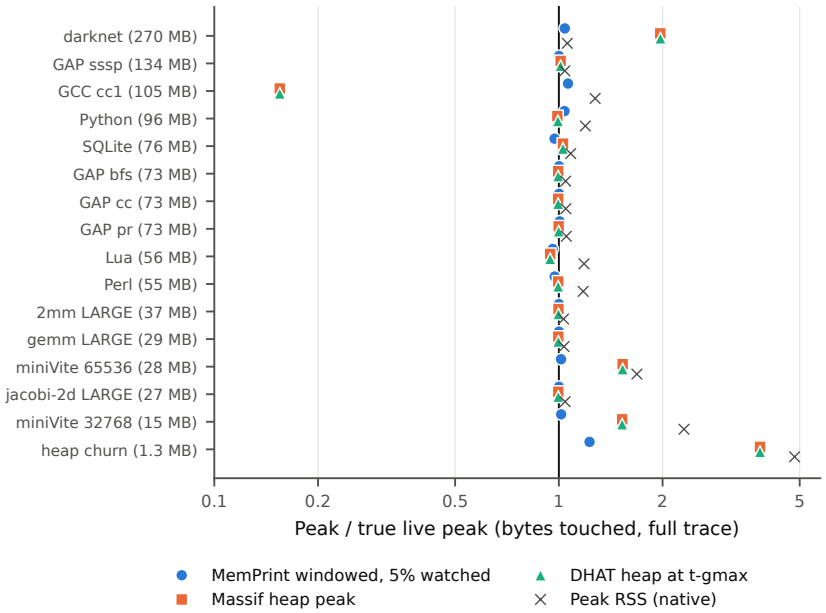


Figure 1: Peaks relative to the true live peak (bytes touched, complete recording, given in brackets): the windowed estimate at 5% watched, Massif's heap peak, DHAT's heap at its global maximum, and the native peak resident memory. Workloads are sorted by true peak; the markers of each workload are spread vertically so that equal values stay visible.

pages. Figure 2 shows the same comparison over time. Massif's heap rises in a step as soon as an array is allocated, whereas the truth rises as the array is touched (2mm fills its second array halfway through the run); it stays at the allocation on darknet and the churn test, and it follows the truth on Lua and SQLite, which touch what they allocate. DHAT reports its maximum at the same moment as Massif.

6 Estimating from sampled references

Our first approach applies the idea of MemPrint at every snapshot: estimate the live footprint at time t from the sampled references made up to t. Table 3 summarizes the estimators we tried, and Figure 3 shows them on the MEDIUM input of 2mm. All results in this section are measured with bytes touched; the results with the start-address footprint differed by at most a few points, because PolyBench reads aligned 8-byte numbers.

6.1 The α model at every snapshot

We fitted the α model of Section 3.2 at every snapshot of the training inputs, using the splitter's bins with frees tracked, and applied it to every snapshot of a sampler run. The estimate has 43% mean error for extrapolation and 31% for interpolation. A bin holds between 0.1% and 10% of the footprint and almost never contains the same address twice. A model that works from one bin therefore cannot tell memory that is used for the first time from memory that is used again. As a result, when the true footprint stops growing, the estimate keeps rising, because every new sampled reference looks like new memory.

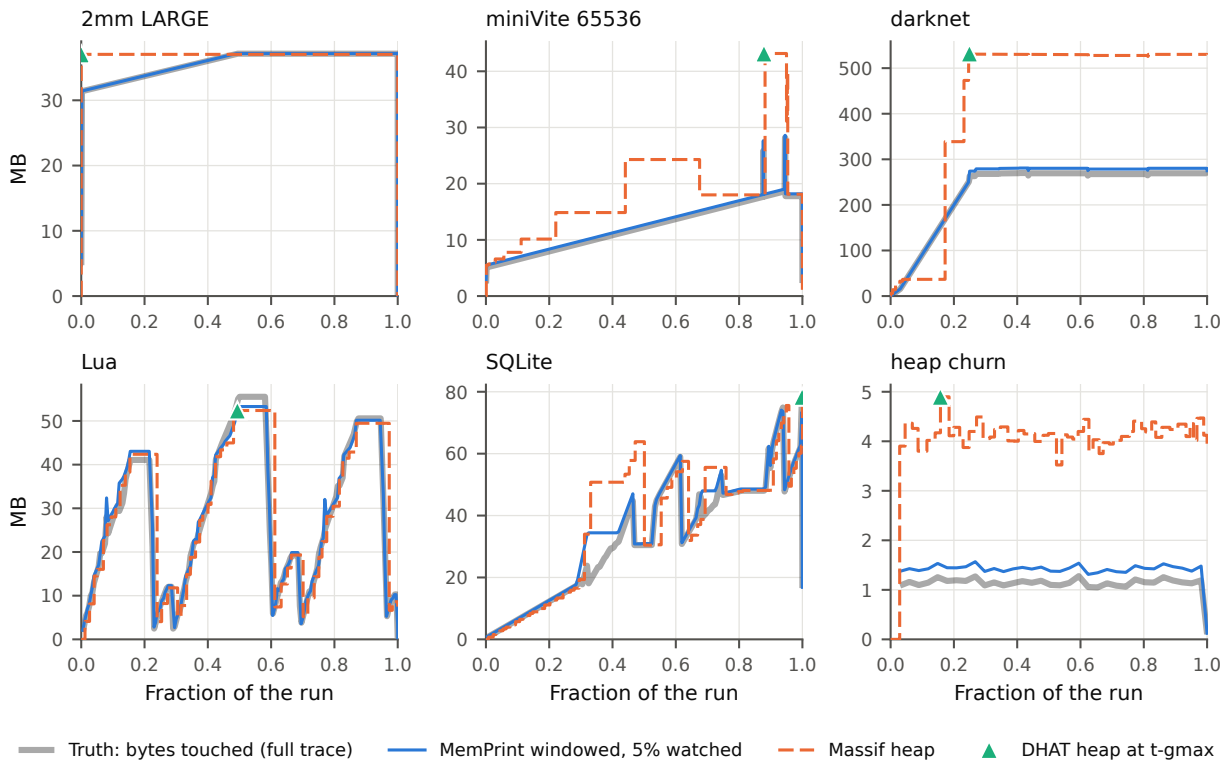


Figure 2: Live footprint over the run: the truth (bytes touched), the windowed estimate at 5% watched, Massif's heap, and DHAT's heap at its global maximum, placed at the moment it occurs. Time is a fraction of the run, counted in memory references for MemPrint and in instructions for Valgrind.

6.2 Estimators of unseen species

The number of distinct addresses that a sample has never seen is an instance of a classic problem in ecology: estimating how many species exist from a sample of animals, some of whose species were never caught [19, 8]. These estimators use *frequency counts*: f_1 is the number of addresses sampled exactly once, f_2 the number sampled exactly twice, and so on. We changed the tool so that every footprint keeps, for each start address, the number of times it was sampled, together with running totals of f_1 to f_4. We also changed the splitter so that it keeps the union of the 20 bins of each interval, which is a 1-in-$k/20$ sample that contains enough repeated addresses.

The Chao1 estimator [9], in its form corrected for small samples, estimates the number of distinct addresses as $S + f_1(f_1 - 1)/(2(f_2 + 1))$, where S is the number of addresses seen. The iChao1 estimator [10] adds the term $\frac{f_3}{4f_4}\max(f_1 - \frac{f_2 f_3}{2f_4}, 0)$, which uses f_3 and f_4 to correct the case in which some addresses are used much more often than others. We convert an estimated number of addresses to bytes by multiplying it by the observed number of bytes per address.

Both estimators stop the estimate from rising on a plateau. For example, the error for jacobi-2d extrapolation falls from 42% to 52% down to 4% on the splitter's unions and to 9% on the sampler (measured with the start-address footprint). However, both are lower bounds when some memory is used once and other memory is used many times, so they underestimate kernels with many addresses written only once by 30% to 60% at moderate sampling rates. At the densest unions they overshoot by 50% to 245%, because f_2 is then much smaller than f_1. Over the four kernels they have 34% mean error for extrapolation, and their error of the peak exceeds 100%.

6.3 An estimator that uses the known sampling rate

Unlike an ecologist, we know the sampling rate p exactly, because it is the number of sampled references divided by the number of references executed, and we also know the number of references executed, T. We use both. We model the number of references made to one address as $1 + X$, where X follows a negative binomial distribution with mean m and shape a. Depending on its shape, this distribution can describe memory that is mostly used once as well as memory that is used many times. If each reference is sampled independently with probability p, the number of times an address appears in the sample has the probability generating function $(q+pz)\,B(z)$, where $q = 1 - p$, $B(z) = ((1 + c) - cz)^{-a}$ and $c = mp/a$. The probability of seeing an address exactly j times is therefore

$$P(j) = q\,b_j + p\,b_{j-1}, \qquad b_j = (1+c)^{-a}\binom{a+j-1}{j}\theta^j, \qquad \theta = \frac{c}{1+c}. \tag{1}$$

Two quantities are known exactly: the total number of references, $U(1 + m) = T$, where U is the number of distinct addresses, and the number of addresses seen, $U(1 - P(0)) = S$. For a given shape a, these two equations determine U and m, which we solve numerically with Brent's root-finding method. We choose the shape among 40 values between 0.02 and 200 by how well $UP(1), \ldots, UP(4)$ and $U(1 - P(0) - \cdots - P(4))$ match f_1 to f_4 and $S - f_1 - \cdots - f_4$, measured by Pearson's chi-square distance.

On the splitter's dense unions the known-rate estimate has 12% mean error for extrapolation and 7% for interpolation, against 44% to 46% and 31% to 34% for Chao1 and iChao1 on the same unions. With the start-address footprint we measured the bias directly: within +1% to −12% at 1 in 5 references, against +53% to +245% for iChao1, and within about 10% to 20% at 1 in 12. At 1 in 50 and sparser, however, f_2 to f_4 are too small to identify the shape, and the error ranges from −90% to +215%. Fixing the shape from the training inputs does not help (40% to 150% error), because the fitted shapes are small (0.02 to 0.08), which indicates a mixture of memory that is used once and memory that is used many times, rather than one smooth distribution.

While evaluating this estimator, we also found that the union of a sampler's bins does not have the sampling rate $1/i$. A sampled reference enters the union only if it is placed in at least one bin, which happens with probability $1 - e^{-\lambda}$, so the union's rate is $(1 - e^{-\lambda})/i$. Our first results had used $1/i$, which overstated the rate by a factor of 1.6.

6.4 A hybrid of the known-rate estimate and a learned correction

The known-rate estimate errs in a consistent direction: it underestimates while addresses have not yet been used again. We therefore learn a correction on the training inputs. Specifically, we fit a ridge regression [21], which is least squares with a penalty on the size of the coefficients (here with weight 1), for $\log(F/\hat{F}_{\text{known}})$ from nine features: how far the known-rate estimate extrapolates beyond the observed footprint, $\log((f_2 + 1)/(f_1 + 1))$, the fraction of sampled references that were new addresses, f_1/S, the sampling rate, the bins' standard deviation, mean footprint and sampling interval, and the fraction of new addresses among the samples of the last five snapshots, from a counter of how often an address entered the sample. We fit the correction on the splitter's unions at the sampling rate closest to that of the run being estimated.

With a dense sampler (1 in 3 references), the hybrid estimate has 6% mean error for extrapolation and 5% for interpolation. At 1 in 25 it has 25% and 12%.

We then tried three further changes at 1 in 25. First, we trained the correction at exactly the sampler's rate, by running the splitter with matching intervals. Second, we made the estimate non-decreasing between frees, using isotonic regression [2] (the closest non-decreasing curve in the least

Table 2: Hybrid estimator at sampling intervals between 3 and 250, correction trained at exactly the sampler's rate and smoothed between frees: mean absolute error over the four kernels in percent (extrapolation / interpolation), and the running time on the MEDIUM inputs relative to a complete recording.

Interval	Mean error	Error of the peak	Error at the peak	Time
3	4.4 / 4.2	3.4 / 7.0	11.2 / 7.9	40%
5	6.4 / 6.1	6.0 / 10.0	17.5 / 14.5	27%
8	15.0 / 9.3	33.1 / 14.0	33.4 / 17.4	18%
12	24.3 / 11.5	64.9 / 22.2	34.3 / 19.4	15%
25	23.5 / 10.0	25.0 / 17.5	37.3 / 26.3	10%
50	30.6 / 15.6	40.6 / 13.6	60.0 / 22.8	7%
100	29.7 / 20.3	17.1 / 29.2	54.3 / 26.7	—
250	44.7 / 18.8	23.8 / 28.6	55.0 / 50.5	—

Table 3: Estimators that work from sampled references, on four PolyBench kernels, with the largest (extrapolation) and middle (interpolation) of the sizes MINI to MEDIUM held out. Values are the mean absolute error over the four kernels, in percent.

		Extrapolation			Interpolation		
Estimator	Sample	Mean	Of peak	At peak	Mean	Of peak	At peak
α model at every snapshot	1/25, 1/50	43	27	61	31	59	44
Chao1	1/12	34	101	54	36	16	28
iChao1	1/12, 1/25	34	107	55	45	17	31
Known rate	1/5	18	43	28	12	39	16
Hybrid	1/3	6	13	11	5	11	8
Hybrid, exact rate, smoothed	1/3	4	3	11	4	7	8
Hybrid, exact rate, smoothed	1/25	24	25	37	10	18	26
Word sampling (Section 7)	1/25	1.6	1.5	1.5	4.0	4.0	4.0

squares sense) on each segment between two frees, because the live footprint can only decrease when memory is freed. Third, we fitted one reuse shape per run instead of one per snapshot. The extrapolation error stayed between 23% and 25% after each change. Replacing the features measured in bytes with ratios that do not depend on the input size did not reduce it either (26%). The remaining error is a bias with input size: at 1 in 25 the largest input of 2mm is 23% low on average and that of jacobi-2d 38% low, mostly in the first half of the run. In 2mm and gemm, each array element is used about N times for an $N \times N$ matrix, so a larger input shows more repeats per address at the same sampling rate than any training input did, and a correction learned on smaller inputs does not apply.

Where the hybrid falls off. Between 1 in 3 and 1 in 25 references the hybrid holds up to 1 in 5 and falls off between 1 in 5 and 1 in 8 (Table 2). At 1 in 5 it is within 3% to 9% mean error and -7% to $+14\%$ of the peak on every kernel and both splits, at 67% of the cost of 1 in 3. At 1 in 8 the peak of 2mm is 56% too high, and at 1 in 12 161%: past its training range, the correction overshoots.

Forecasting. We also tried to predict the rest of a footprint curve from its beginning. We stretched each training input's curve in time and in size to match the observed beginning, and averaged the stretched curves with weights that decrease with how badly each one fitted. The

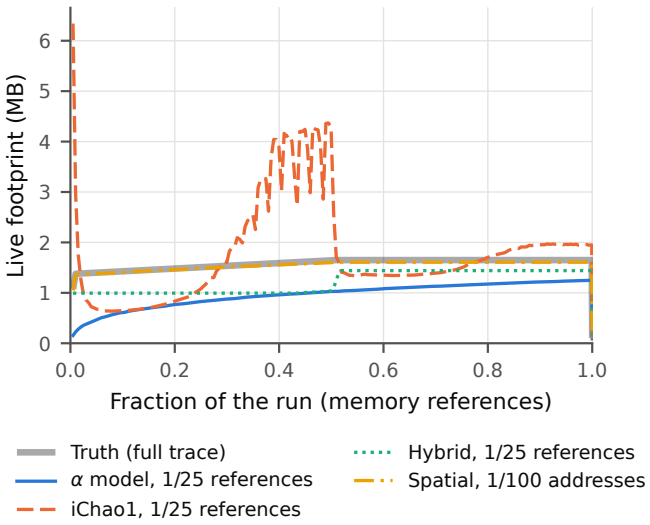


Figure 3: Live footprint of 2mm MEDIUM (held out from training) over the run, against estimates from 1-in-25 samples of references and from a 1-in-100 sample of words. The word sample follows the truth closely, whereas the estimates from sampled references overshoot, undershoot or lag.

predicted peak was within about 10% to 20% for interpolation but unreliable for extrapolation, and we did not pursue forecasting further.

7 Sampling words

All the estimators of Section 6 fail for the same reason. An address that is referenced r times appears in a sample of references with probability $1 - (1 - p)^r$, which depends on r, and r is unknown and grows with the input size. Sampling memory instead of references removes that dependence, because a byte is then in the sample with a probability that does not depend on how often it is used.

7.1 Method

In the tool's *spatial* mode (`-mode spatial`), an aligned 8-byte word $w = \lfloor a/8 \rfloor$ is *selected* when $h(w \oplus \sigma) < 2^{64}/i$. Here $\oplus$ is the bitwise exclusive or, σ is a 64-bit number (a salt) derived from the run's random seed, and h is the final mixing function of the splitmix64 random number generator [44], which scrambles the bits of its input so that nearby words, or words at regular distances, give unrelated results. For every reference that covers a selected word, the tool records the bytes of the reference that lie inside selected words.

Each byte is selected with probability $1/i$, independently of how the program uses it. Let $F_S(t)$ be the number of selected bytes touched and live at time t. Every byte of the true footprint contributes to F_S with probability $1/i$, so the expected value of $F_S(t)$ is $F(t)/i$, and

$$\hat{F}(t) = i\, F_S(t) \tag{2}$$

is an *unbiased* estimate of $F(t)$, meaning that its average over many choices of the salt equals the truth. It is the Horvitz–Thompson estimator [22] for a sample in which every element has the same probability of being included [12]. No model and no training are needed. The number of

selected words follows a binomial distribution, so for U touched words, the standard deviation of the estimate relative to the truth is about $\sqrt{(1-1/i)\,i/U} \approx 1/\sqrt{U/i}$. For example, 200,000 words sampled at 1 in 100 give about 2%.

The unit matters. A first version selected 64-byte chunks, which is 8 times fewer units for the same memory, and its error on PolyBench was about $\sqrt{8} \approx 2.8$ times larger (7.4% instead of 2.8% at 1 in 100 on the largest inputs). With the start-address footprint the unit is the start address, which for 8-byte numbers gives the same precision as words.

To report an error bar, the tool also splits the selected words into 20 *buckets* by the value of $h(w \oplus \sigma) \bmod 20$. Each bucket is an independent 1-in-$20i$ sample, so $20\,i\,F_{\text{bucket}}$ is an independent estimate of F, and the standard deviation of the 20 bucket estimates divided by $\sqrt{20}$ is a standard error of $\hat{F}$.

Implementation. The selection test is an inlined test routine that receives the reference's address and size, adds one to the running thread's reference counter, and returns whether any word the reference covers is selected. A reference of up to 16 bytes covers at most three words, which the inlined test checks; for wider references, whose size is known when the instruction is instrumented, a routine with a loop checks every word. A selected reference calls an action routine that records it immediately in the footprint and in its bucket, while holding a lock shared by all threads. Because selected references are rare, recording them immediately costs little, and it keeps the order of references and frees across threads exact, because the tool also applies frees immediately under the same lock. The time of a snapshot is the sum of the threads' reference counters.

We first recorded selected references through trace buffers, as the splitter does, which brought back the ordering problem of Section 4.5. We then held each free back until every thread had processed all the references it had made before the free, using vector clocks, which record for each thread how many events of every other thread it has seen. This approach was correct, but miniVite frees memory constantly while its MPI helper threads rarely fill a buffer, so a run at 8192 vertices took 7474 s. Recording selected references immediately took 28 to 40 s for the same run, and the 8-thread test of Section 4.5 then gives 12.03 to 12.27 MB, against a truth of 12.09 MB.

7.2 Results

PolyBench. With the largest and middle of the sizes MINI to MEDIUM held out, as in Section 6, word sampling has 1.6% and 4.0% mean error at 1 in 25 words, 2.8% and 4.9% at 1 in 100, 2.9% and 4.1% at 1 in 250, and 5.3% and 16.0% at 1 in 1000 (Figure 4). The error follows the binomial prediction. The largest sizes (130,000 to 200,000 words) stay within about 5% down to 1 in 1000, whereas the middle sizes (about 28,000 words) need 1 in 250 or denser, because at 1 in 1000 only about 28 of their words are selected. For the largest sizes, the true footprint lies within two standard errors of the estimate in 99% of snapshots at 1 in 25 and 1 in 100 and in 75% at 1 in 250, against 95% expected, because a standard error computed from 20 buckets is itself uncertain.

miniVite and threads. With frees tracked, the live peak of miniVite is 3.8, 4.8 and 6.3 MB at 1024, 4096 and 8192 vertices, the same at every thread count. Word sampling has 0.3% to 6.8% mean error at every rate from 1 in 25 to 1 in 1000 with 1, 4 and 16 threads, and the error of the peak is at most 3.6% at 1 in 100 or denser and at most 7.7% at 1 in 1000. The error bars are calibrated across salts but not within a run. Over 20 runs of miniVite with 4096 vertices and different salts, at a given snapshot the truth lies within two standard errors in 94% (1 in 100) and 95% (1 in 250) of the runs, and the standardized errors have a standard deviation of 1.03 and 1.10 across runs and a mean of -0.09 and $+0.14$. A single run, however, keeps the same selected words,

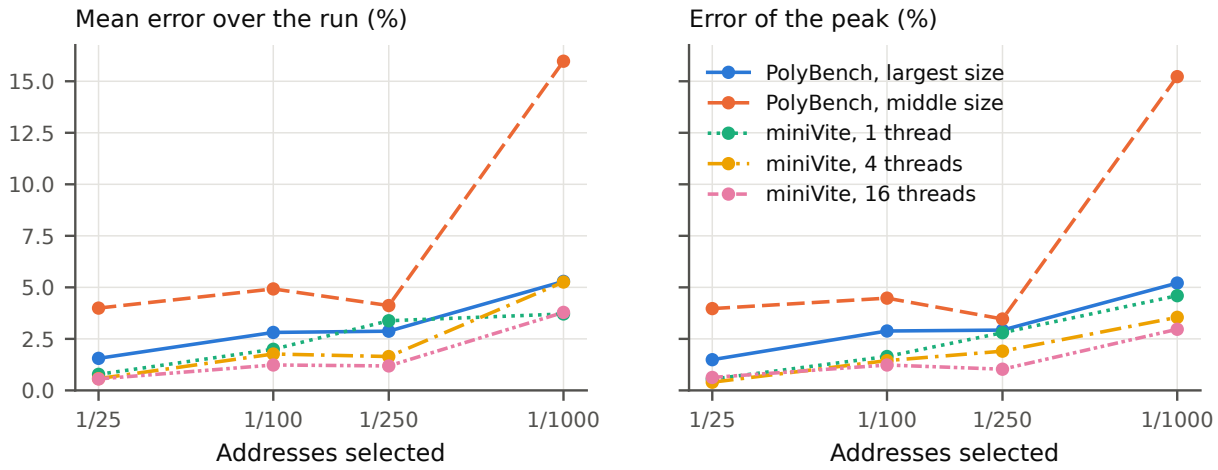


Figure 4: Word sampling error against the fraction of words selected. PolyBench values are the mean over 2mm, gemm, jacobi-2d and atax for the held-out largest and middle of the sizes MINI to MEDIUM. miniVite values are the mean over 1024, 4096 and 8192 vertices.

so its error persists from one snapshot to the next (lag-one correlation 0.82), and the share of its snapshots within two standard errors is nearly all or nothing (10% to 100% over the 20 runs). At 8192 vertices a run takes 25 to 41 s, against 175 to 187 s for a complete recording and 0.16 to 0.22 s natively.

Cost on large inputs. On the LARGE PolyBench inputs, word sampling at 1 in 100 takes 284 s for 2mm, 271 s for gemm and 344 s for jacobi-2d, against 8.0, 5.1 and 11.3 s natively, which is an overhead of 30× to 53×. Pin without a tool takes 8.2, 5.4 and 11.5 s, so almost all of the cost is the selection test on every reference. The test checks one or two words for most references, against one start address for the start-address footprint, which took 211 to 236 s.

8 Windowed sampling

Word sampling still runs the selection test on every reference, which costs 30× to 53× native time on the LARGE PolyBench inputs, whereas Pin without a tool costs almost nothing. For long programs, therefore, the test itself is the obstacle, not the sampling rate. *Windowed sampling* (options `-window W` and `-period P`) runs the selection test only during *windows* of W references at the start of every P references, so that a fraction W/P of the run is *watched*. Between windows it runs only things that cost little and that still observe the whole run. Figure 5 gives an overview, and the rest of this section describes each part.

8.1 Counting time between windows

Between windows the tool must still count references, because time is measured in references, and windows start and end at given times. Counting each reference separately would cost almost as much as the selection test. Instead, when Pin translates a basic block, the tool counts the block's memory references, which is the number of memory operands that the block's instructions read plus the number that they write, and it inserts one inlined call at the start of the block that adds that number to the running thread's counter. The same call compares the counter with a threshold

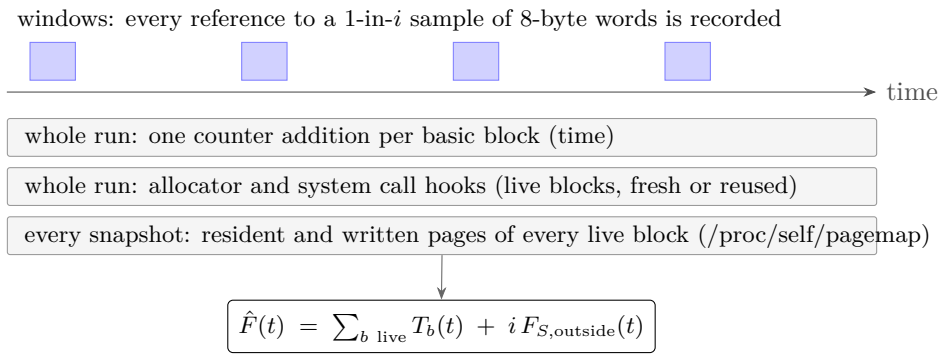


Figure 5: Windowed sampling. References are checked only inside windows (shaded). Time, live blocks and page information are tracked for the whole run. T_b is the touched bytes of block b (resident bytes for a fresh block, bytes on written pages for a reused block), and $F_{S,\text{outside}}$ is the selected bytes outside all blocks.

and, only when the threshold is reached, calls a longer routine that advances time for all threads (Section 8.2).

On miniVite with one thread, this count matches the splitter's count of every reference to within 0.0001% over 1.78 billion references. The two counts differ in two cases. An instruction with a REP prefix, which repeats a string operation many times, counts once rather than once per repetition, and a basic block that is left before its end, for example because of a signal, still counts all its references.

Each thread's counter sits in its own 64-byte memory area, which is the unit in which processors cache memory, so that threads do not slow each other down by writing into the same cached unit.

8.2 Switching between watching and not watching

When a thread's counter passes its threshold, the thread takes the shared lock, adds up all threads' counters to get the current time, writes any snapshot that is due, and opens or closes a window if the time has passed a window's start or end. It then sets its next threshold s references ahead, where

$$s = \max\Big(10^4,\ \tfrac{1}{8}\min(W,\ P - W,\ N)\Big)\,, \tag{3}$$

so that every thread checks the time several times within the shortest of a window, a gap between windows and a snapshot interval, but not so often that the lock becomes expensive. The first version used 10^3 instead of 10^4, and a run with 1000-reference windows then spent most of its time taking the lock.

When a window opens or closes, the instrumentation must change, because inside a window every reference needs the selection test, and outside a window it does not. The tool calls `PIN_RemoveInstrumentation`, after releasing the lock, which discards all translations, so that Pin translates code again, with the new instrumentation, the next time it runs. However, discarding translations does not change code that is already running. A loop keeps jumping back to the start of its own translation, so it keeps running the old instrumentation until it leaves the loop. In 2mm, a window that had closed kept the selection test running for the rest of a loop nest, so a run that should have watched almost nothing took 70 s instead of 19 s, and windows did not start or stop on time.

The tool therefore counts window changes in a shared counter, and each thread remembers the last change it has acted on. When a thread checks the time and finds a change it has not acted on, it restarts at its current instruction with `PIN_ExecuteAt`, using the machine state that Pin passes

to the routine, so that Pin translates the code it is running again. Threads check the time every s references also while running old code, because the counting call is present in both versions of the code. A reference that old code records after a window has closed is discarded.

Discarding all translations has a cost that grows with the amount of code a program runs, which Section 9.6 reduces with trace versions. For most programs it is small, but GCC's compiler runs a lot of code, and with 20 windows, which is 40 changes, it took 658 to 963 s, against 101 to 122 s with nothing watched and 182 s watching the whole run (Section 9.5).

8.3 Tracking live blocks

In windowed mode the allocator hooks of Section 4.1 are always on, and the tool keeps every live block in an ordered map from start address to size, so that it can find the block that contains a given address. It also records memory that the program maps directly. When an `mmap` system call that maps anonymous memory succeeds outside an allocation call, the tool adds the mapping as a block, and when `munmap` unmaps part of such a block, the tool removes that part and keeps the rest. When `realloc` grows or shrinks a block in place, the tool keeps the block and changes its size.

The tool classifies each block as *fresh* or *reused*. A block is fresh when its pages were mapped for it, which is the case for an anonymous mapping made outside the allocator, and for an allocation during which the allocator called `mmap` or `mremap`. The tool knows the latter because it watches system calls and knows which thread is inside an allocation call. By default, the GNU C allocator serves requests of 128 KB or more by mapping new memory [16], so large arrays are usually fresh. All other blocks are reused, because the allocator takes them from memory it already owns, which may have been used by blocks that were freed earlier.

8.4 Page residency

A fresh block's pages are untouched when the block is created, and a page becomes resident at its first read or write. The number of a fresh block's bytes that lie on resident pages is therefore the number of bytes the block has touched, rounded to whole pages, whether or not a window is open.

The tool reads residency from `/proc/self/pagemap` [45], a file through which Linux reports one 64-bit entry per page of the process. In an entry, bit 63 is set if the page is resident, bit 62 if it has been moved to swap space, and bit 55 if it has been written since the last reset of that bit (Section 8.5). At every snapshot the tool visits the live blocks in address order, reads the entries of their pages in chunks of 512 pages so that neighboring blocks share reads, and for each block adds up the bytes of the block that lie on resident or swapped pages. At a block's first and last page it counts only the bytes inside the block. Pin's C library does not provide the `pread` function, so the tool uses `lseek` followed by `read`.

Our machine enables transparent huge pages [47], which let the operating system back a 2 MB region with one large page at its first touch. A single touch can then make a whole 2 MB region resident. In a test that wrote 16 MB of a 64 MB block, pagemap reported 18.4 MB resident with huge pages on. The tool therefore turns off huge pages for the traced process with `prctl(PR_SET_THP_DISABLE)`, through the raw system call because Pin's C library does not provide `prctl`. With huge pages off, the same test reports exactly 16 MB, and reading another 8 MB adds exactly 8 MB, because a read of untouched memory also makes its page resident.

8.5 Written pages of reused blocks

The residency of a reused block can include pages that earlier blocks at the same addresses touched, because memory that a program frees stays resident and the allocator hands it to later blocks. In a test that keeps 64 heap blocks live, replaces the oldest one at every step, and writes only the first quarter of each new block (Section 9.3), residency overestimates the live footprint by 249%.

The tool therefore measures a reused block by the pages written since the block was allocated, using the kernel's soft-dirty bits [46]. Writing the character `4` to the file `/proc/self/clear_refs` clears the soft-dirty bit of every page of the process, and after that, bit 55 of a page's pagemap entry is set if the page has been written. Memory that the allocator returns is not initialized, so a program writes heap memory before it reads it, and the pages a reused block has written since its allocation are the pages it has touched. Each reused block keeps one mark per page, and at every snapshot the tool sets the mark of each of its pages whose soft-dirty bit is set. The touched bytes of a reused block are its bytes on marked pages.

To clear the soft-dirty bits, the kernel removes write permission from every page, so the next write to each page causes a fault, in which the kernel sets the bit and restores the permission. Each clear therefore costs one fault for each page that is written afterwards, and the tool clears the bits only at three kinds of events. Before every clear, it first sets the marks of every live reused block from the current bits, so that no write is lost.

1. At the entry of an allocation call of 16 KB or more. Bits that earlier blocks set on the new block's pages are then cleared before the block exists. The clear happens at the entry rather than at the exit because `calloc` sets the new memory to zero and `realloc` copies the old block's contents inside the call, and those writes belong to the new block. Smaller blocks share pages with other blocks, so clearing for them would add cost without making their measurement exact.

2. Around the `brk`, `mmap` and `mremap` system calls. When the kernel creates or grows a memory region, it marks the whole region as soft-dirty, so that all its pages read as written until the next clear, whether or not they were written. The allocator grows the heap with `brk`, and before we handled this case, the written bytes of the churn test jumped between 1.3 MB and 3.7 MB from one snapshot to the next. The tool now sets the marks at the system call's entry and clears the bits at its exit.

3. Every `-dirty_every` references, by default every ten snapshots.

A block whose pages are all marked cannot change any more, so the tool skips it when it sets marks before a clear. On miniVite with one thread, which clears the bits 8509 times, this reduced the running time from 138 s to 86 s.

8.6 Word sampling inside windows

Inside windows the tool selects 1 in i words, as in Section 7. The windows see memory outside every block, which covers the stack, global variables and memory mapped from files, and which pages cannot attribute to a block.

Density, under the start-address footprint. With the start-address footprint, residency and written pages measure bytes, but the footprint counts overlapping accesses more than once, so the tool multiplied each block's touched bytes by a *density*, the footprint divided by the bytes covered. Windows then selected 64-byte chunks, so that they saw, for some part of each block,

both the footprint and the bytes covered, and a block's density was measured once its chunks covered 64 KB/i; otherwise a reused block used the density of all reused blocks and a fresh block used 1. In miniVite's reused blocks the density was about 1.45 for most of the run. This was the weakest part of the estimate. A block's density accumulates over a run, because every new access pattern, another width or another offset, adds start addresses to memory already covered, so a window measures a lower density than the whole run has: on GAP breadth-first search, 1.02 at 1% watched, 1.08 at 5% and 1.19 at 20%, against 1.36 over the whole run, which left its peak 26% too low at 5% watched. With bytes touched there is no density, which is the second reason we changed the definition.

8.7 The estimate

At every snapshot the tool writes one row to a separate output file (Appendix B), which includes the estimate

$$\hat{F}(t) = \sum_{b \text{ live}} T_b(t) + i\, F_{S,\text{outside}}(t), \tag{4}$$

where T_b is the touched bytes of block b (resident bytes for a fresh block, bytes on written pages for a reused block), and $F_{S,\text{outside}}$ is the number of selected bytes outside every block, which only windows see. The row also contains every part of the estimate, so that other combinations can be evaluated without running the program again.

8.8 Windows opened by large allocations

With `-window_alloc N` (by default 1 MB), an allocation of at least N bytes also opens a window at once, at most one per period on top of the periodic windows, so that at most twice the periodic fraction is watched. A large allocation is usually followed by the program filling the block, and with the start-address footprint a window opened then measured the block's density. A periodic window that starts inside such a window extends it.

With the start-address footprint these windows mattered. On miniVite with 65536 vertices, a 25 MB fresh block allocated at 88% of the run was inside no periodic window and counted with density 1.00 instead of 1.28, while a 9 MB reused block allocated at 94% took the density of the other reused blocks, 1.44 instead of 1.00; the two errors cancelled at the peak (-2.6%). Windows opened at both allocations measured both blocks exactly. With bytes touched there is no density to measure, and these windows change the error by less than one point (Section 9).

8.9 Approaches that did not work

We arrived at (4) after several approaches that failed, which we list in the order in which we tried them.

1. **The windows' sample alone.** Multiplying the selected bytes by i misses every address that is touched only between windows. On 2mm LARGE at 1% watched it has 47% mean error.

2. **A model per allocation site.** We estimated a block from the selected bytes that windows had seen in it, and a block that no window had yet seen from the fraction touched in earlier blocks allocated at the same place in the program. This estimate also missed every touch made between windows, and it had 29% to 47% mean error on miniVite at 1% watched and 8% to 25% at 5%.

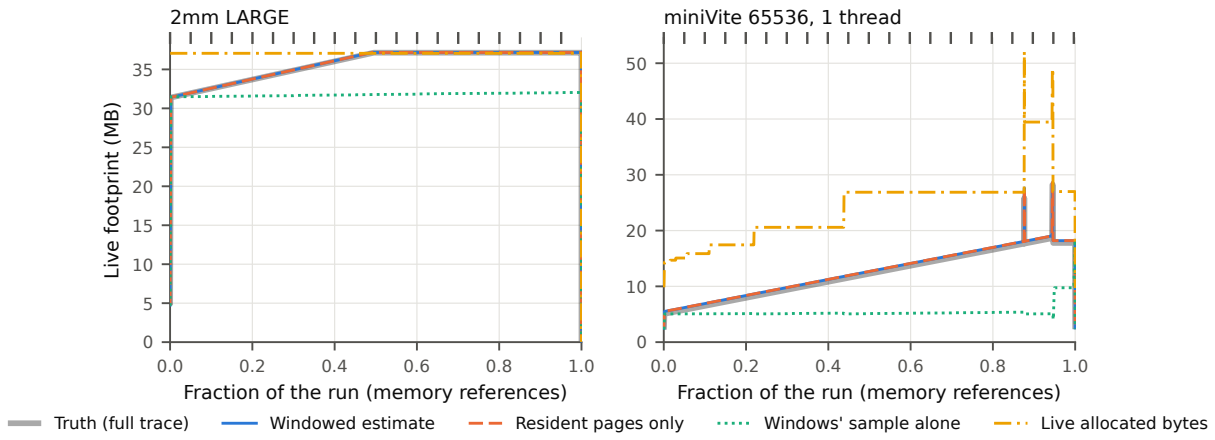


Figure 6: Live footprint over the run with 5% of the run watched (20 windows, marked at the top) and 1 in 100 words selected. On 2mm LARGE the windowed estimate and residency alone coincide with the truth.

3. **Residency for all blocks.** Residency is exact for fresh blocks but overestimates reused blocks that sit on memory used before, by 249% on the churn test.

4. **A density from sampled pages.** Dividing the selected footprint by the resident bytes of the pages that contained a selected address was noisy, and one run ended with a density of 2.7, because a page with few touched addresses enters the calculation only when one of them is selected.

5. **Densities measured in windows.** Under the start-address footprint, one density for all reused blocks, and later one per block, left peaks up to 15% too low on miniVite and up to 66% too low on the programs of Section 9.4, because density accumulates over the whole run (Section 8.6).

9 Evaluation of windowed sampling

We compare windowed runs with a complete recording of the same input and thread count. All runs select 1 in 100 words and use about 20 windows per run (the period is about one twentieth of the run), with periodic windows only unless we say otherwise, and we did not tune any setting for any workload. We compare (4) with three simpler estimates: *residency alone*, which is the resident bytes of all blocks plus the windows' selected bytes outside blocks, without written pages; the windows' selected bytes alone ($i\,F_S$); and the bytes of all live blocks, which we call the *allocated bytes*.

9.1 PolyBench

Every PolyBench array is allocated by mapping new memory, so all blocks are fresh and residency alone gives the exact curve. The windowed estimate is within 0.0% to 0.1% mean error and within 0.1% at the peak at every watched fraction, including none (Figure 6, left). The windows' sample alone has 0.1% to 10.9% mean error at 5% watched and 10.9% to 46.7% at 1%, because it misses memory that is written once and then not used while a window is open. The allocated bytes have 0.4% to 4.5%.

Table 4: Windowed estimate on miniVite, mean error / error of the peak in percent, 1 in 100 words, about 20 windows.

Input, threads	Nothing watched	1%	5%	20%	Everything
65536 vertices, 1 thread	10.1 / −3.8	2.8 / +1.1	3.9 / +1.5	3.7 / +1.4	3.6 / +1.4
65536 vertices, 4 threads	9.8 / −3.7	2.4 / +0.9	3.6 / +1.3	4.2 / +1.6	3.8 / +1.4
32768 vertices, 1 thread	14.7 / −6.9	1.0 / +0.5	3.3 / +1.5	5.3 / +2.5	4.9 / +2.3
32768 vertices, 4 threads	14.6 / −6.9	1.8 / +0.8	4.0 / +1.8	5.3 / +2.5	5.7 / +2.7

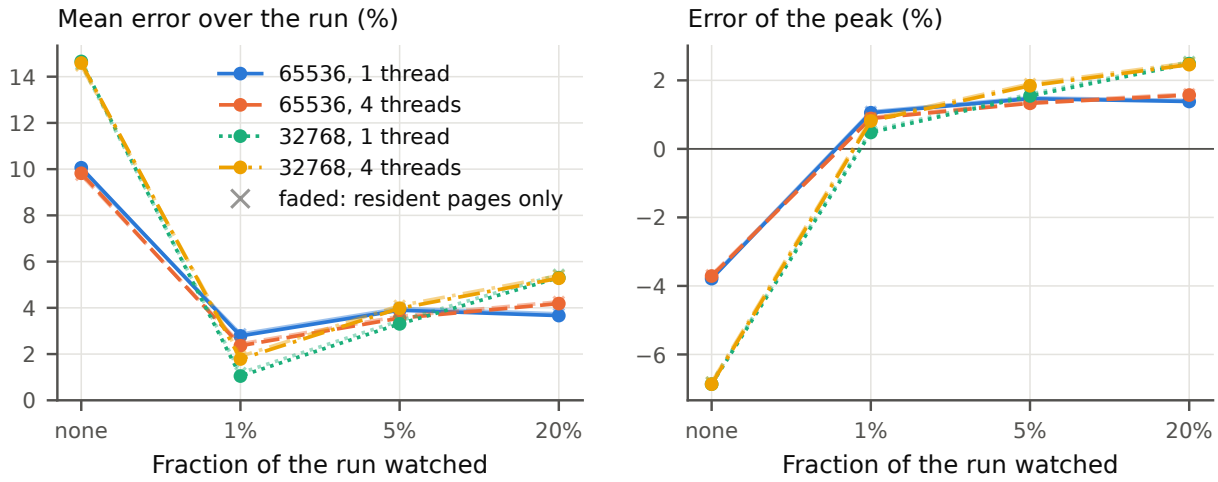


Figure 7: miniVite: windowed estimate error against the fraction of the run watched, with 1 in 100 words. Faded lines are residency alone, without written pages.

9.2 miniVite

miniVite's footprint grows in reused heap blocks for the whole run, and it allocates far more than it touches, so the allocated bytes overestimate it, with 107% to 164% mean error (Figure 6, right). Table 4 and Figure 7 show the windowed estimate. With 5% watched, it has 3.3% to 4.0% mean error and +1.3% to +1.8% error of the peak on both inputs and both thread counts. Watching matters only for memory outside blocks, which only windows see: with nothing watched, the peak is 4% to 7% too low, and 1% watched is enough. Watching more does not help, and the estimate rises by one to two points between 1% and 20% watched, as whole pages and selected words outside blocks accumulate. Selecting 1 in 25 words instead of 1 in 100 changes the results by less than one percentage point, and one and four threads agree within one point. Residency alone is as good as the estimate on miniVite, because its reused blocks are written all over.

With the start-address footprint, the same runs had 1.7% to 17.1% mean error and −0.4% to −15.2% error of the peak at 1% to 20% watched, and about 25% too low a peak with nothing watched, because the density had to be measured in windows.

Windows opened by large allocations. miniVite makes 5 to 7 allocations of 1 MB or more that open windows. With them the results change by about one point or less (for example 3.6% / +1.4% instead of 3.9% / +1.5% at 5% watched with one thread on 65536 vertices). PolyBench and the churn test open no such windows.

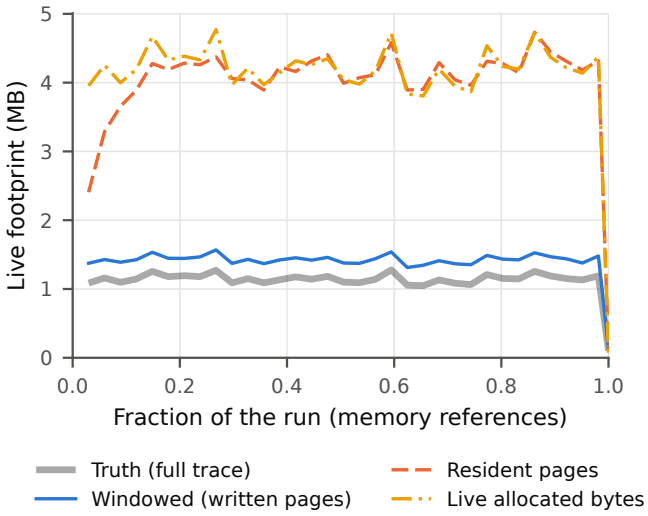


Figure 8: Heap churn with a quarter of each block written, 5% of the run watched. Resident pages include memory that freed blocks touched, whereas the pages written since each block's allocation follow the truth to within whole pages.

9.3 Heap churn

The heap-churn test keeps 64 live heap blocks of 16 to 112 KB, which is below the allocator's threshold for mapping new memory, so all blocks come from memory that the allocator reuses. At every step it frees the oldest block, allocates a new one, writes the first quarter (or half) of the new block, and reads the written part of one other live block. Its live footprint peaks at 1.3 MB when a quarter of each block is written and at 2.5 MB when half is written.

Residency alone overestimates the footprint by 249% (mean error) when a quarter of each block is written and by 91% when half is written, because the pages of freed blocks stay resident and are handed to new blocks (Figure 8). Measuring reused blocks by their written pages reduces the error to 24% and 13%. The error is about the same at every watched fraction (23% to 26% when a quarter is written), because written pages, not windows, carry the estimate. The remaining error comes from whole pages. Each block's written part ends inside a 4 KB page, which adds up to 4 KB per block, or about 0.25 MB for 64 blocks.

9.4 Programs that reuse their heap

Table 5 shows the windowed estimate on the GAP kernels, darknet, the interpreters, SQLite and GCC, with allocation windows on. Lua, Perl and SQLite keep 84% to 100% of their live memory in reused heap blocks (44 to 73 MB at the peak, in 24,000 to 650,000 blocks), whereas GAP, darknet, Python and GCC keep most of theirs in fresh blocks: GAP's and darknet's arrays are large, Python's small-object allocator takes 256 KB regions from the operating system, and GCC's garbage collector maps its pages. In every program, the residency of the reused blocks exceeds their written pages by at most 1.2 MB at any point: the blocks are reused, but each new block is written all over. Forcing large blocks of GAP and darknet onto the heap, with the environment variable `MALLOC_MMAP_THRESHOLD_`, moved 5 to 86 MB into reused blocks, and residency still exceeded the written pages by at most 1 MB.

Every peak is within −5.4% to +6.7% at every watched fraction, including none. The programs

Table 5: Windowed estimate on programs that reuse their heap, mean error / error of the peak in percent, 1 in 100 words, allocation windows on. Single thread unless stated.

Program	True peak	Allocated bytes	Nothing	1%	5%	Everything
GAP bfs	72.8 MB	201.9 / −0.3	5.0 / −0.3	3.1 / −0.0	2.1 / +0.2	2.3 / +0.2
GAP pr	72.5 MB	143.8 / −0.0	3.8 / −0.0	1.8 / +0.4	1.7 / +0.5	1.5 / +0.5
GAP cc	72.8 MB	198.7 / −0.3	4.9 / −0.3	3.0 / −0.0	1.9 / +0.1	2.0 / +0.2
GAP sssp	133.8 MB	183.3 / +1.3	3.6 / −0.2	2.4 / +0.1	2.3 / +0.1	2.3 / +0.0
GAP pr, 4 threads	72.6 MB	337.6 / +34.6	8.3 / −0.0	11.7 / +0.4	6.4 / +0.5	4.1 / +0.5
darknet	269.9 MB	153.2 / +96.9	3.1 / +4.0	3.2 / +4.6	3.2 / +4.1	3.2 / +4.1
Python	95.7 MB	3.4 / +3.0	3.4 / +2.8	4.1 / +3.8	3.5 / +3.8	4.9 / +4.0
Lua	55.5 MB	5.8 / −5.6	5.9 / −5.4	6.6 / −4.8	6.2 / −3.7	11.8 / +5.2
Perl	54.9 MB	1.7 / −2.3	1.9 / −3.5	1.9 / −2.9	1.9 / −2.7	1.7 / −0.6
SQLite	76.2 MB	16.9 / −2.9	6.4 / −0.7	8.5 / −2.4	8.2 / −2.5	8.4 / −0.8
GCC `cc1`	105.4 MB	11.5 / +8.3	3.6 / +3.5	8.1 / +6.4	8.5 / +6.7	8.9 / +6.6

that touch what they allocate (GAP, Lua, Perl, SQLite) are estimated almost as well by their allocated bytes, at the peak; darknet, which touches half of what it allocates, is not (+97%). On GCC and Lua, watching moves the peak up by three to ten points, as the windows' selected bytes outside blocks exceed what is live there; we have not found which memory that is. With the start-address footprint, the peaks of the same programs were 5% to 66% too low at 5% watched, mostly because of density, and also because the truth then counted the allocator's bookkeeping (Section 4.2).

9.5 Cost

Section 10 compares every method of this report with the final tool; the measurements here are of the tool as described up to this section.

Table 6 and Figure 9 report running times measured one at a time on an otherwise idle machine, one run per configuration, except where a run was repeated (the table then shows the median). On PolyBench LARGE, windowed sampling at 5% watched costs 3.5× (jacobi-2d) and 3.6× (2mm) native time, and 4.5× to 12× on gemm, whose running time varied between 23 and 61 s over three runs (the table shows the median), against 30× to 53× for word sampling of every reference, and with nothing watched it costs 1.9× to 2.3×, which is mostly the cost of the per-block counter. Each further percent watched adds about 0.8% of the cost of word sampling over that floor. On miniVite with 65536 vertices it costs 14.9× with one thread and 18.1× with four at 5% watched, against 34× and 37× for word sampling, and 12.6× and 16.7× at 1%, which is as accurate.

The cost on miniVite is higher for three reasons. First, Pin itself costs about 1.8×, because Pin translates the MPI library's start-up code once and the run is short. Second, miniVite makes 8 million `malloc` and `free` calls at 65536 vertices, each of which runs the allocator hooks and takes the lock, and on miniVite with 16384 vertices, turning the allocator hooks off saved 7 s of 22 s. Third, soft-dirty tracking cost about 14% at 5% watched in an earlier measurement, and 4% to 14% on PolyBench. The complete recordings that provided the truth took 3116 to 4080 s, but they ran at the same time as other runs, so they are not directly comparable.

On the programs of Table 5, measured with several runs at once and therefore only roughly, Python, Lua, Perl and SQLite took 24 to 91 s with nothing watched, 52 to 216 s at 5% watched and 95 to 364 s watching everything, against 450 to 1684 s for their complete recordings. GCC took 101 to 122 s with nothing watched and 182 s watching everything, but 658 to 963 s with windows,

Table 6: Running time in seconds, one run each, 1 in 100 words, about 20 windows.

Workload	Native	Pin, no tool	Word sampling	Windowed, fraction watched			
				none	1%	5%	20%
2mm LARGE	8.0	8.2	284	15.7	18.1	29.0	67.7
gemm LARGE	5.1	5.4	271	11.5	13.9	48.1	56.6
jacobi-2d LARGE	11.3	11.5	344	21.6	25.0	39.8	93.3
miniVite 65536, 1 thread	6.4	11.7	216	67.4	81.2	96.1	121
miniVite 65536, 4 threads	6.0	11.1	221	84.1	101	109	136

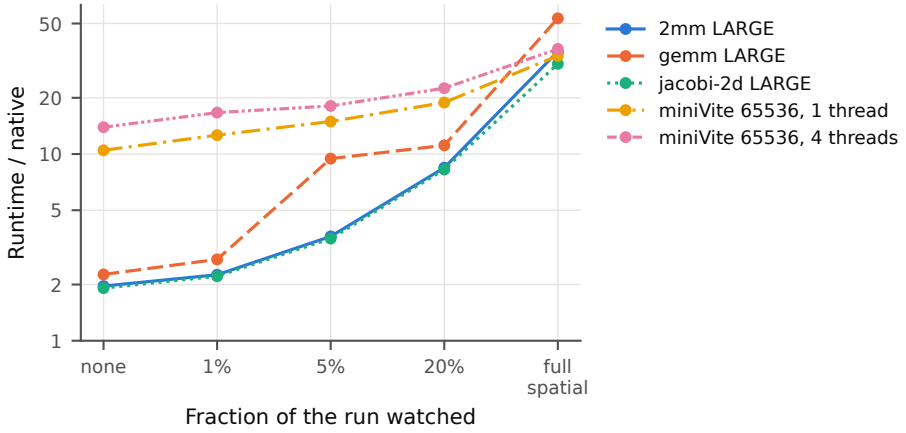


Figure 9: Running time relative to native time against the fraction of the run watched, with word sampling of every reference for comparison (logarithmic scale).

because every window change makes Pin translate all of its code again (Section 8.2).

9.6 Fewer windows, cheaper switching and partial pages

With bytes touched the windows matter less than before, so we tried four changes, evaluated against the same complete recordings (Table 7).

No windows. With `-pages_only 1`, the tool never opens a window. Memory outside blocks comes from the resident pages of the program's writable image sections (`.data` and `.bss` of every loaded image, which Pin reports when it loads the image) and of the main thread's stack (the 8 MB below the top of the stack mapping, read once from `/proc/self/maps`); the stacks of other threads are anonymous mappings, which the tool already tracks as blocks. This brings miniVite with 32768 vertices from −6.9% to −3.5% at the peak with nothing watched, and every peak is within −5.4% to +5.4%. A pages-only run that never discarded its translations ran gemm and 2mm 8 times slower than one that did, for a reason we have not found (not the snapshots and not soft-dirty tracking), so the tool discards them once after the first snapshot.

Partly used pages. A touched page inside a block that borders an untouched page is probably only partly used. Counting such a page as half used (`-edge_correction`, now the default) removes the churn test's error from whole pages (+22.4% to +1.5% at the peak with a quarter of each block written, +12.2% to +1.4% with half) and changes the other results by at most a point.

Table 7: Windowed estimate with the partial-page correction, mean error / error of the peak in percent, 1 in 100 words, periodic windows; "before" is the tool of Section 9 with nothing watched.

Run	Before, nothing	Pages only	1%	5%
miniVite 65536, 1 thread	10.1 / −3.8	5.0 / −1.9	1.8 / +0.7	2.9 / +1.1
miniVite 32768, 1 thread	14.7 / −6.9	7.4 / −3.5	0.9 / +0.4	2.1 / +1.0
heap churn, 25% touched	22.8 / +22.2	3.0 / −1.2	1.1 / +1.6	1.4 / +1.5
heap churn, 50% touched	12.4 / +12.0	1.6 / −0.3	1.2 / +1.7	1.1 / +1.4
GAP bfs	5.0 / −0.3	4.5 / −0.3	3.0 / −0.1	1.8 / +0.2
darknet	3.1 / +4.0	3.1 / +4.0	3.1 / +4.0	3.1 / +4.0
Python	3.4 / +2.8	4.4 / +3.8	5.6 / +3.7	6.2 / +4.0
Lua	5.9 / −5.4	5.9 / −5.4	4.7 / −4.9	6.1 / −4.1
Perl	1.9 / −3.5	1.9 / −3.4	1.8 / −2.9	1.8 / −2.1
SQLite	6.4 / −0.7	6.7 / −2.7	8.3 / −2.5	8.1 / −2.1
GCC `cc1`	3.6 / +3.5	6.4 / +5.4	7.7 / +5.9	8.2 / +6.5

Table 8: Running time in seconds, one run at a time on a machine shared with other jobs (so only roughly comparable with Table 6).

Workload	Native	Pages only	1%, auto	5%, auto	5%, flush
2mm LARGE	9.6–13.4	23.2	20.2	35.5	45.4
gemm LARGE	5.3–5.6	11.8	15.3	25.9	27.6
jacobi-2d LARGE	11.9–12.5	22.6	28.0	44.9	47.7
miniVite 65536, 1 thread	6.8–7.3	69.6	88.7	108.0	110.2
miniVite 65536, 4 threads	6.4–6.8	85.8	114.0	137.4	123.5
GCC `cc1`	1.7–1.9	81.7	137.5	152.7	706.3
Lua	3.3–3.7	34.4	57.8	56.5	58.2

The allocator's slack. On Lua, 5.9 of the 6.2 MB that the windows select outside blocks lies within 16 bytes past the end of a live block: string and copy functions read past the requested size into the slack that the allocator adds, in 650,000 small blocks. The truth counts these bytes, only windows see them, and so Lua's peak rises from −5% to +5% as more of the run is watched. Measuring blocks over their whole chunk (`-block_slack 1`) fixes the peak but counts slack that is never read, which makes Lua's mean error 14% instead of 6%, so it is off by default. On GCC, by contrast, the memory outside blocks is far from any block (globals and stack), and the pages of its blocks overcount by about 8 MB, because its garbage collector uses parts of pages.

Cheaper switching. Pin's trace versions let two instrumented versions of the code coexist: a test at each trace entry compares a tool register, which the time check sets to the window state, and moves to the other version when they differ, so nothing is discarded. On GCC at 5% watched this takes 153 s instead of 706 s, but the test at every trace entry made 2mm with nothing watched take 63 s instead of 16 s. With `-window_switch auto`, the default, the tool flushes until one window change retranslates more than 10,000 traces and then switches to versions for good: GCC retranslates about 47,000 traces per change, Lua 1300, miniVite 210 and 2mm 11, so only GCC switches. Table 8 gives the running times.

Table 9: MPI runs, ranges over the processes. Footprints in MB; estimates as mean error / error of the peak in percent where a complete recording exists, else the estimated peak.

Run	True peak	Peak RSS	Allocated	Pages only	1% watched
LULESH 20, 8 processes	9.0–9.1	20.2–21.8	18.0–18.1	20.7–22.4 / −8.7 to −9.4	12.9–15.5 / −0.8 to −2.1
miniVite 131072, 4 processes	16.4–17.1	33.4–34.4	30.9–31.5	9.2–11.9 / −5.4 to −6.0	3.2–5.4 / −2.0 to +0.2
LULESH 50, 8 processes	—	105.3–107.2	100.3–100.4	90.5–90.6	92.4–92.6
miniVite 524288, 4×4 threads	—	76.4–79.5	88.7–90.9	58.6–60.8	59.3–61.6

9.7 Many MPI processes

To see how the estimate behaves with several processes and on runs too long to record completely, we ran LULESH 2.0, a hydrodynamics proxy application, with 8 MPI processes, and miniVite with 4 processes, built against the Spack OpenMPI 5.0.5. Each process runs under its own Pin and writes its own files. Two medium runs (LULESH with 20^3 elements per process and 100 iterations, miniVite with 131072 vertices) were also recorded completely; two long ones (LULESH with 50^3 elements and 500 iterations, 156 s natively; miniVite with 524288 vertices and 4 threads per process, 146 s) were not, and we compare them with each process's peak resident memory. Table 9 gives the results.

Watching 1% of the run, every process's peak is within 2.1% of its truth, and the processes agree with each other within a point. Without windows, the estimate is 5% to 9% low at the peak, because MPI's own memory, such as its shared-memory segments, which are mappings of files rather than anonymous memory, lies outside the blocks and outside the image sections that the pages-only mode measures, so only windows see it. For MPI programs, at least 1% should therefore be watched. The true footprint is 0.41 to 0.50 of the resident memory on the medium runs, because resident memory also counts the loader, the MPI library's code and data, and whole pages. On the long runs the windowed peak is 0.77 to 0.88 of the resident memory and below it on every process; resident memory is only an upper bound, so it cannot check the estimate more closely. Relative to native time, long runs are cheaper: 4.6× (LULESH) and 5.6× (miniVite) without windows, and 10.3× and 5.2× at 1% watched, against 18× to 34× for the 3 s medium LULESH run, because the fixed cost of starting Pin and translating the MPI library is spread over a longer run.

9.8 Other allocators

The tool now also handles an allocator that replaces the GNU C library's, such as jemalloc 5.3 and tcmalloc (gperftools 2.18), which we load with `LD_PRELOAD`. The first image other than the C library that defines `malloc` is taken as the allocator, and all of its code counts as the allocator's for the exclusion of its bookkeeping (Section 4.2). Four problems had to be solved. First, Pin did not list jemalloc's exported functions at all, so the tool reads their addresses from the library's dynamic symbol table and creates the routines itself; such a routine extends to the next symbol that Pin knows, over other functions (jemalloc's `memalign` covered `realloc`), so the tool takes each function's size from the ELF symbol tables and hooks only the returns inside it. Second, both allocators leave their slow paths by jumping into another function, sometimes one without a symbol, so the tool catches a call's exit at the first of three places: a return of a jump target executed with the stack pointer of the outermost call's entry, dispatched by the kind of call that was entered (jemalloc's `malloc` and `realloc` jump to the same function); the instruction at the return address of a direct call to an allocation function; and return addresses learned at run time, because programs built without PLT stubs, such as the system's `sqlite3`, call through the global offset table: each allocation call records its return address, and the code there is translated again,

Table 10: Lua and SQLite under three allocators: true peak and peak RSS in MB, estimates as mean error / error of the peak in percent, 1 in 100 words.

Allocator	Program	True peak	Peak RSS	Pages only	1% watched
GNU C library	Lua	55.5	—	5.9 / −5.4	4.7 / −4.8
jemalloc	Lua	51.7	93.2	7.6 / +1.9	8.3 / +2.8
tcmalloc	Lua	55.3	69.4	6.1 / +1.3	7.4 / −0.9
GNU C library	SQLite	76.2	—	6.8 / −2.7	8.5 / −2.5
jemalloc	SQLite	81.0	144.1	10.7 / −6.4	10.0 / −7.0
tcmalloc	SQLite	79.2	112.1	2.2 / −0.9	8.5 / −2.7

with the exit, before the call returns. Third, at `free` the tool releases the block's usable size, which it asks the allocator for, so that bytes read past the requested size are freed, and blocks whose allocation was missed are freed too. Fourth, the bugs were only visible as leaks: before the size fix, jemalloc's `realloc` left 16,468 of 20,000 blocks live in a unit test.

Table 10 gives the results on Lua and SQLite. Every peak is within −7.0% to +2.8%. The true peaks depend on the allocator, because size classes and layouts differ. tcmalloc remains partial: a program that allocates and frees 600,000 small blocks ends with 5.75 MB live, against 1.70 MB under the GNU allocator and 2.15 MB under jemalloc, and Lua with 5.3 MB instead of 0.3 MB, because writes to tcmalloc's metadata that are made outside its own code are counted and never released. Allocators that manage regions inside the program, such as Python's small-object allocator and GCC's garbage collector, are still tracked only through the regions that they map.

10 Overhead of every method

The costs reported so far were measured with different builds of the tool and under different loads. To compare all the methods of this report on equal terms, we ran each of them with the final tool on eight programs: 2mm, gemm and jacobi-2d at the LARGE size, miniVite with 65536 vertices and 1 or 4 threads, Lua, SQLite and GCC's `cc1`. We measured the wall time and the peak resident memory of the whole process (program, Pin and tool) with `/usr/bin/time`. Runs that take a few minutes or less were repeated three times and we report the median; runs differed by a few percent, more for the short SQLite and GCC runs. Up to six runs (briefly ten) shared a 32-core machine, which was also running another user's jobs, so the slowdowns, which are relative to the median of three native runs made alone, include some interference: native runs made in the same mix were 1.1× to 1.3× slower than alone, and 2mm, which is limited by memory bandwidth, 1.8×. Table 11 and Table 12 give the results, and Figure 10 shows both. Late in the experiment another user's job filled the machine's memory and swap; the complete recordings of miniVite overlapped that period and may be slowed by it, and we stopped three of our runs and repeated them after the job had finished (Lua's complete recording ran on an otherwise quieter machine).

Pin and per-reference instrumentation. Pin without a tool costs 1.1× to 1.6× on PolyBench and 2.2× on miniVite, but 2.8× to 13× on the short interpreter, database and compiler runs, where translating code dominates: GCC's 1.6 s run executes a lot of code once. Instrumenting every reference, even with an analysis routine that only counts, costs 13× to 54×. This is the floor for every method that looks at every reference: MemPrint's splitter and sampler, the complete recording and word sampling.

Complete recordings. MemPrint's splitter, which keeps 260 subsampled footprints (13 intervals with 20 bins each) for its whole-run model, costs 347× to 866× native time (540× to 781× on PolyBench) and needs 3.5 to 21 GB of memory: 580× to 760× the program's own on PolyBench, whose arrays it records word by word in every bin, and 73× to 125× elsewhere. The complete recording that we use as the truth, with one bin, freed memory removed and snapshots, costs 134× to 398× and 0.6 to 3.5 GB, 13× to 44× the program's memory. Either is affordable only for small inputs.

Estimators from sampled references. MemPrint's sampler (Section 6) costs 29× to 109× at 1 in 100 references and 52× to 132× at 1 in 25, above the instrumentation floor, because every selected reference updates hash sets for each of its bins. Its memory is the largest of the sampling methods: 2.7 to 3.8 GB on PolyBench LARGE at 1 in 100, and at 1 in 25 on 2mm and gemm it exceeded about 5 GB and Pin aborted with an out-of-memory error, although the machine had memory to spare. The estimators that need training (the α model, the hybrid) add the complete recordings of smaller inputs: for 2mm, recording the seven training sizes from MINI to MEDIUM took 128 s, against 0.1 s natively, and fitting and evaluating every estimator took 190 to 220 s per program on one core. The unseen-species estimators need no training.

Word sampling. Word sampling without windows (Section 7) costs 27× to 68×, close to the instrumentation floor, and needs 3× to 10× the program's memory, mostly Pin's own: it keeps only the selected words.

Windowed sampling. With pages only, windowed sampling costs 2.0× to 3.2× on PolyBench, 12× to 14× on miniVite and 13× to 17× on Lua and SQLite; at 1% watched, 2.5× to 3.4×, 14× to 18× and 22× to 52×. On PolyBench every further percent watched costs about as much as word sampling on that fraction of the run, so 20% watched costs 10× to 13×. On the other programs the cost of watching is smaller than the fixed cost, which comes from the allocator hooks (miniVite, Lua, SQLite) and from translating code (GCC: 50× with pages only, 85× to 108× with windows, and 395× at 5% if every window change discards all translations, against 99× with the automatic switch to trace versions). SQLite's run is 0.9 s natively, so the fixed cost of starting Pin and the tool weighs heavily: 17× with pages only and 52× at 1%, where every window change discards the translated code and Pin translates it again (Section 8.2). The memory is 2.1× to 3.8× the program's on PolyBench and 2.9× to 16× elsewhere, close to Pin's own (1.9× to 5.8×): blocks, pages and selected words take little.

Heap profilers. Massif costs 5.6× to 23× and DHAT 15× to 74×, both with less than 2.2× the memory (4.4× for DHAT on Lua). Windowed sampling at 1% is cheaper than DHAT on every program except SQLite and GCC, and cheaper than Massif on PolyBench and miniVite, but costs 3× to 8× more than Massif on the interpreters and the compiler, where the allocator hooks and Pin's translation dominate. Massif and DHAT measure allocations, however, not the memory touched (Section 5.3).

Summary. The floor set by per-reference instrumentation (13× to 54×) separates the methods: every method that examines every reference costs at least that much, and the complete recordings and MemPrint's splitter 4× to 33× more. Windowed sampling is the only method below that floor, by 7× to 10× on PolyBench and 1.6× to 1.8× on miniVite at 1% watched. On the short interpreter, database and compiler runs it is above the floor, because the allocator hooks and code

Table 11: Running time as a multiple of the native time (first row, in seconds, median of three runs alone). Medians of three runs where repeated. OOM: Pin aborted at about 5 GB.

	2mm	gemm	jacobi-2d	miniVite 1t	miniVite 4t	Lua	SQLite	GCC
Native (s)	7.4	5.0	11.2	6.4	6.0	2.5	0.9	1.6
Pin, no tool	1.6	1.2	1.1	2.2	2.3	2.8	3.9	13
Per-reference instrumentation only	26	28	24	26	28	13	27	54
MemPrint splitter (13 intervals × 20 bins)	665	781	540	863	866	347	554	502
Complete recording (truth)	299	291	285	341	398	134	217	210
Reference sampling, 1 in 100	54	79	42	44	46	29	47	109
Reference sampling, 1 in 25	OOM	OOM	86	81	87	52	107	132
Word sampling, 1 in 100	48	59	36	37	54	27	49	68
Windowed, pages only	3.2	2.5	2.0	12	14	13	17	50
Windowed, 1% watched	3.4	3.2	2.5	14	18	22	52	85
Windowed, 5% watched	6.0	5.6	4.0	18	20	22	54	99
Windowed, 20% watched	13	13	10	23	26	25	62	108
Windowed, 5%, flushing	4.8	5.4	4.0	18	20	29	53	395
Massif	5.6	6.1	7.9	20	23	6.6	9.8	11
DHAT	16	17	15	32	34	44	43	74

Table 12: Peak resident memory in MB of the process (program, Pin and tool).

	2mm	gemm	jacobi-2d	miniVite 1t	miniVite 4t	Lua	SQLite	GCC
Native	36	29	27	46	48	63	79	128
Pin, no tool	70	62	60	261	266	125	170	747
Per-reference instrumentation only	78	77	74	348	353	144	202	776
MemPrint splitter (13 intervals × 20 bins)	21,131	17,420	20,502	3,456	3,459	7,818	8,516	9,984
Complete recording (truth)	750	611	728	764	621	1,197	3,466	2,946
Reference sampling, 1 in 100	2,785	2,662	3,805	700	698	387	666	2,024
Reference sampling, 1 in 25	OOM	OOM	5,050	850	846	596	1,322	2,975
Word sampling, 1 in 100	121	105	102	452	444	247	355	1,263
Windowed, pages only	77	69	67	400	411	265	227	1,289
Windowed, 1% watched	105	97	100	447	455	278	269	1,728
Windowed, 5% watched	114	103	102	462	479	285	288	1,858
Windowed, 20% watched	115	103	100	477	491	303	317	1,990
Windowed, 5%, flushing	113	104	100	436	442	277	287	1,587
Massif	60	53	51	90	90	137	111	223
DHAT	64	56	54	98	101	275	120	236

translation, not the references, dominate there; a longer run of the same programs would weigh these fixed costs less.

11 Discussion and limitations

Why each part is needed. Each part of (4) corrects one failure that we observed. Residency records first touches whether or not a window is open, which the windows' sample alone cannot do. Written pages remove the pages that a reused block inherits from freed blocks, which residency alone cannot do. The windows see memory outside every block, which pages cannot attribute. With bytes touched, that is all: the density, which was the weakest part of the estimate, is gone, and with it most of the reason to watch more than 1% of a run.

The definition of the footprint. The start-address footprint of MemPrint measures the access pattern as well as the memory, by up to 3.4× on SQLite, and a heap profiler measures allocation, which can be 2× to 4× the memory touched. The bytes touched are bounded by the resident

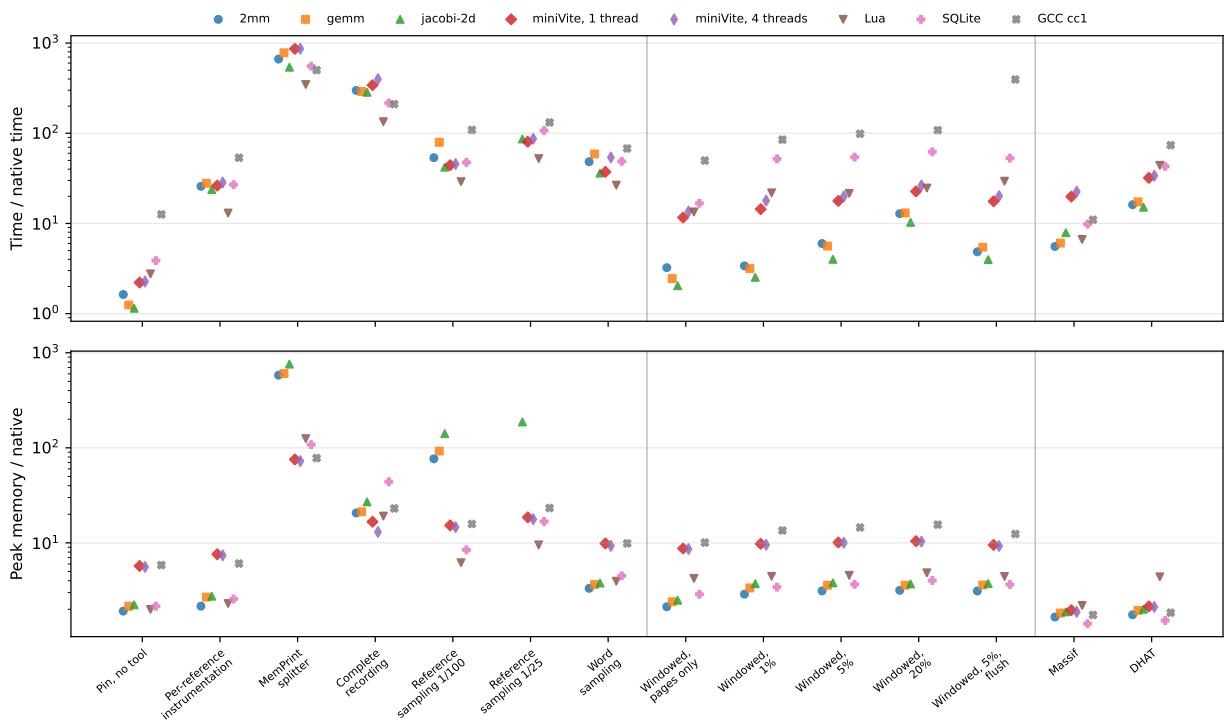


Figure 10: Running time and peak memory of every method relative to the native run, one marker per program. The vertical lines separate the methods that examine every reference, windowed sampling, and Valgrind's heap profilers.

memory, which our validation confirmed on every workload. The tool keeps the start-address footprint for MemPrint's whole-run model, whose outputs do not change.

Limitations of written pages. A limitation of written pages is that they miss memory that a reused block only reads, which is rare for heap memory because the allocator does not initialize it, and that they are exact only to the page, which adds up to 4 KB per block and dominates for programs with many small blocks that are only partly touched (23% on the churn test). Counting a page at the border of a block's untouched part as half used reduces that to 1.5% (Section 9.6), but it is a rule of thumb, not a measurement. It does not help darknet (+4%), whose partly touched pages lie at the ends of fresh blocks, or GCC (+6%), whose collector uses parts of its pages. Blocks smaller than 16 KB do not clear the soft-dirty bits when they are allocated, so they can inherit marks from writes made since the last clear. With several threads, writes that other threads make between the entry and the exit of a `brk` or `mmap` call are cleared without being recorded.

Limitations of the allocator handling. The tool recognizes the GNU C library's allocator by the names of its functions, which needs the library's symbol table, and it reads the allocator's chunk size field to release a whole chunk. It also handles jemalloc and tcmalloc (Section 9.8), although tcmalloc's metadata writes made outside its own code leave a few megabytes live, and other replacement allocators are untested. Programs that manage memory inside large regions of their own, as Python's small-object allocator and GCC's garbage collector do, are tracked only at the level of the regions that they map.

Other limitations. Memory outside blocks is seen only in windows, except for the writable sections of the loaded images and the main thread's stack, whose resident pages the pages-only mode measures; MPI's shared memory is not among them, so MPI programs need windows (Section 9.7). The tool does not track `mremap` calls made outside `realloc`, `madvise`, or the growth and shrinking of the stack. A window change discards all translated code until one change has retranslated more than 10,000 traces, after which the tool keeps two versions of the code instead; the version test at every trace entry then costs time for the rest of the run. Address and word selection depend on absolute addresses, which move when the tool library changes size, because the operating system places the program's mappings around the tool's, so spatial and windowed outputs are not byte for byte identical across builds of the tool, whereas splitter and sampler outputs are. Soft-dirty tracking requires a Linux kernel built with soft-dirty support, and the tool falls back to residency for reused blocks if `/proc/self/clear_refs` cannot be written.

Generality of the evaluation. We evaluated three scientific applications, four graph kernels, a neural network, three interpreters, a database, a compiler and one synthetic test, mostly on one input each, and two of the programs under two other allocators. With up to 8 MPI processes, the estimate at 1% watched stays within 2.1% of the peak (Section 9.7), but the longest runs could be checked only against resident memory. It remains unclear how the windowed estimate behaves on long programs whose footprint changes in phases shorter than a window period.

12 Related work

Measuring memory use. Valgrind [33] is a binary instrumentation framework, and its heap profilers Massif [41] and DHAT [32] record every allocation and report the amount of heap memory reserved over time, at a high cost and without measuring which reserved bytes are used; DHAT also counts the accesses to each byte offset of small blocks. Pin [28] and DynamoRIO [7] are binary instrumentation frameworks on which tools that record every memory access can be built, as MemPrint [31] does. The working set model [13] defines the memory that a program needs as the pages it used during a recent interval of time, which is a page-level, time-limited counterpart of the live footprint.

Hardware sampling of memory accesses. Intel's Processor Event-Based Sampling (PEBS) and ARM's Statistical Profiling Extension (SPE) let the processor record the address of a sampled memory access at low cost [34, 30]. MemGaze [26] analyzes traces of sampled loads, and HeMem [39] uses PEBS to decide which pages to place in fast memory. These approaches sample accesses, as the MemPrint sampler does, so an estimate of the footprint from them faces the bias of Section 6: frequently used memory is far more likely to be seen.

Page-level tracking by the operating system. Thermostat [1] samples pages and observes accesses to them by changing their page table entries, and DAMON [36] monitors how often memory regions are accessed in the Linux kernel by checking the accessed bits of sampled pages. Linux reports residency through `/proc/pid/pagemap` [45] and written pages through soft-dirty bits [46], which tools that save and restore processes use to find the pages changed since a previous save. We use residency and soft-dirty bits to measure first touches between windows, to the page, and we use sampled references to see what pages cannot attribute.

Sampling locations in caches and simulation. Simulating only a subset of cache sets, or only the addresses chosen by a hash, is an established way to make cache simulation cheaper [25]. SHARDS [48] keeps the references whose address hashes below a threshold to build miss ratio curves for storage caches from a small fixed fraction of the addresses, which is the same selection as our word sampling. Counter stacks [49] use probabilistic counters to estimate distinct elements and miss ratio curves for storage traces. Reuse distance analysis [29, 5], StatCache [4] and StatStack [15] estimate cache behavior from sampled reuse, rather than the footprint over time.

Sampling over time. PinPoints [37] and SimPoint [42] choose representative regions of a run, SMARTS [50] simulates many short, regularly spaced samples of a run in detail, and instrumentation sampling for datacenter applications [11] turns instrumentation on and off over time, as our windows do. These methods estimate quantities that each region shows by itself, such as instructions per cycle. A footprint depends on the first touch of every address, which can happen outside every sampled region, which is why windowed sampling needs residency and written pages between windows.

Estimating unseen elements. Good [19] introduced the estimation of the probability of unseen species from frequency counts, and Chao1 [9] and iChao1 [10] estimate the number of unseen species [8]. Our known-rate estimator adapts these ideas to a sample whose rate is known exactly. Word sampling uses the Horvitz–Thompson estimator [22, 12], and the MemPrint sampler uses Poisson bootstrap weights [14, 20].

13 Next steps

We propose the following next steps, in order of priority.

1. **Measure the slack and the used part of pages from the sample.** Lua's bytes in its blocks' slack (−5%) and GCC's partly used collector pages (+6%) are the remaining errors; the word sample could estimate both per block, in place of the half-page rule.

2. **Explain the slowdown of pages-only runs** that never discard their translated code (Section 9.6).

3. **Reduce the fixed cost of the allocator hooks and of soft-dirty clears.** On miniVite the allocator hooks account for about a third of the windowed running time. Instrumenting the allocator in Pin's probe mode, which patches the function's entry instead of translating code, or recording allocation events in a per-thread log that is processed at snapshots instead of under a lock, would lower both.

4. **Allocators that manage regions of their own.** Recognize Python's small-object allocator and GCC's garbage collector, so that their objects are tracked as blocks, and find tcmalloc's metadata writes made outside its code (Section 9.8).

5. **MPI memory without windows.** Measure the resident pages of MPI's shared-memory segments directly, so that the pages-only mode, which is 5% to 9% low on MPI programs (Section 9.7), needs no windows there either.

6. **Evaluate across nodes.** Run with many MPI processes on several nodes, where communication buffers grow with the number of processes, and record one medium run completely to check it.

7. **Combine with hardware sampling.** PEBS or SPE could replace the per-block counter and the word sample at even lower cost, with the known-rate correction of Section 6.3 for their bias toward frequently used memory.

8. **Revisit forecasting.** Windowed sampling makes the beginnings of long runs cheap to record, which may make it practical to predict the rest of a run from the phases seen so far.

14 Conclusion

The footprint of a program is best counted as the bytes it touches: MemPrint's start-address definition counts overlapping accesses more than once, by up to 3.4×, and heap profilers count allocations, which can be several times the memory touched. Estimating the bytes in use over time from a random sample of memory references fails at sparse sampling rates, because such a sample cannot separate memory used for the first time from memory used again. Sampling memory words instead removes that difficulty and needs no training, but it still checks every reference. Checking references only during short windows, while tracking allocations, resident pages and, for reused memory, written pages for the whole run, keeps the estimated peak within 0.1% on PolyBench LARGE, within 1.8% on miniVite, and within −3.7% to +6.7% on ten other programs at 5% of the run watched, at about 3.5× native time on PolyBench and 15× to 18× on miniVite, and less at 1% watched, which is as accurate. Counting pages at the border of a block's untouched part as half used, and switching to two versions of the code for programs with a lot of code, make 1% watched the best setting: within −4.9% to +5.9% at the peak on every program. The estimate holds with 8 MPI processes, within 2.1% of every process's peak, as long as some of the run is watched, and under the jemalloc and tcmalloc allocators.

References

[1] Neha Agarwal and Thomas F. Wenisch. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 631–644, 2017.

[2] R. E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference under Order Restrictions: The Theory and Application of Isotonic Regression.* John Wiley & Sons, 1972.

[3] Scott Beamer, Krste Asanović, and David Patterson. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[4] Erik Berg and Erik Hagersten. Fast data-locality profiling of native execution. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '05, page 169–180, New York, NY, USA, 2005. Association for Computing Machinery.

[5] Kristof Beyls and Erik H. D'Hollander. Reuse distance-based cache hint selection. In *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, Euro-Par '02, page 265–274, Berlin, Heidelberg, 2002. Springer-Verlag.

[6] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.

[7] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. An infrastructure for adaptive dynamic optimization. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 265–275, 2003.

[8] John Bunge and M. Fitzpatrick. Estimating the number of species: A review. *Journal of the American Statistical Association*, 88(421):364–373, 1993.

[9] Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984.

[10] Chun-Huo Chiu, Yi-Ting Wang, Bruno A. Walther, and Anne Chao. An improved nonparametric lower bound of species richness via a modified Good–Turing frequency formula. *Biometrics*, 70(3):671–682, 2014.

[11] Hyoun Kyu Cho, Tipp Moseley, Richard Hank, Derek Bruening, and Scott Mahlke. Instant profiling: Instrumentation sampling for profiling datacenter applications. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–10, 2013.

[12] William G. Cochran. *Sampling Techniques.* John Wiley & Sons, 3rd edition, 1977.

[13] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[14] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[15] David Eklov and Erik Hagersten. StatStack: Efficient modeling of LRU caches. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 55–65, 2010.

[16] Free Software Foundation. *The GNU C Library Reference Manual*, 2024. Section "Memory Allocation Tunables" (`M_MMAP_THRESHOLD`), `https://www.gnu.org/software/libc/manual/`.

[17] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI)*, pages 97–104, 2004.

[18] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[19] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953.

[20] James A. Hanley and Brenda MacGibbon. Creating non-parametric bootstrap samples using Poisson frequencies. *Computer Methods and Programs in Biomedicine*, 83(1):57–62, 2006.

[21] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[22] D. G. Horvitz and D. J. Thompson. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952.

[23] Roberto Ierusalimschy, Luiz Henrique de Figueiredo, and Waldemar Celes. Lua—an extensible extension language. *Software: Practice and Experience*, 26(6):635–652, 1996.

[24] Intel Corporation. *Pin 3.30 User Guide*, 2023. `https://software.intel.com/sites/landingpage/pintool/docs/98830/Pin/doc/html/index.html`.

[25] R. E. Kessler, Mark D. Hill, and David A. Wood. A comparison of trace-sampling techniques for multi-megabyte caches. *IEEE Transactions on Computers*, 43(6):664–675, 1994.

[26] Ozgur O. Kilic, Nathan R. Tallent, Yasodha Suriyakumar, Chenhao Xie, Andrés Marquez, and Stephane Eranian. Memgaze: Rapid and effective load-level memory trace analysis. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 484–495, 2022.

[27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25*, 2012.

[28] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[29] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970.

[30] Samuel Miksits, Ruimin Shi, Maya Gokhale, Jacob Wahlgren, Gabin Schieffer, and Ivy Peng. Multi-level memory-centric profiling on arm processors with arm spe. In *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W '24, page 996–1005. IEEE Press, 2025.

[31] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[32] Nicholas Nethercote and Julian Seward. *DHAT: a dynamic heap analysis tool*. Valgrind user manual, `https://valgrind.org/docs/manual/dh-manual.html`.

[33] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.

[34] Aleix Roca Nonell, Balazs Gerofi, Leonardo Bautista-Gomez, Dominique Martinet, Vicenç Beltran Querol, and Yutaka Ishikawa. On the applicability of pebs based online memory access tracking for heterogeneous memory management at scale. In *Proceedings of the Workshop on Memory Centric High Performance Computing*, MCHPC'18, page 50–57, New York, NY, USA, 2018. Association for Computing Machinery.

[35] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.1*, 2020.

[36] SeongJae Park, Yunjae Lee, and Heon Y. Yeom. Profiling dynamic data access patterns with controlled overhead and quality. In *Proceedings of the 20th International Middleware Conference Industrial Track*, pages 1–7, 2019.

[37] H. Patil, R. Cohn, M. Charney, R. Kapoor, A. Sun, and A. Karunanidhi. Pinpointing representative portions of large intel itanium programs with dynamic instrumentation. In *37th International Symposium on Microarchitecture (MICRO-37'04)*, pages 81–92, 2004.

[38] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. `https://sourceforge.net/projects/polybench/`.

[39] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. HeMem: Scalable tiered memory management for big data applications and real NVM. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 392–407, 2021.

[40] Joseph Redmon. Darknet: Open source neural networks in C. `http://pjreddie.com/darknet/`, 2013–2016.

[41] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at `http://valgrind.org/docs/manual/ms-manual.html`.

[42] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 45–57, 2002.

[43] SQLite. `https://www.sqlite.org`.

[44] Guy L. Steele, Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 453–472, 2014.

[45] The Linux Kernel Documentation. Examining process page tables (/proc/pid/pagemap), 2024. `https://www.kernel.org/doc/html/latest/admin-guide/mm/pagemap.html`.

[46] The Linux Kernel Documentation. Soft-dirty PTEs, 2024. `https://www.kernel.org/doc/html/latest/admin-guide/mm/soft-dirty.html`.

[47] The Linux Kernel Documentation. Transparent hugepage support, 2024. `https://www.kernel.org/doc/html/latest/admin-guide/mm/transhuge.html`.

[48] Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient MRC construction with SHARDS. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*, pages 95–110, 2015.

[49] Jake Wires, Stephen Ingram, Zachary Drudi, Nicholas J. A. Harvey, and Andrew Warfield. Characterizing storage workloads with counter stacks. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 335–349, 2014.

[50] Roland E. Wunderlich, Thomas F. Wenisch, Babak Falsafi, and James C. Hoe. SMARTS: Accelerating microarchitecture simulation via rigorous statistical sampling. In *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, pages 84–95, 2003.

A Options of the Pin tool

Table 13 lists the options of `memprint_trace`. The tool is built against the Pin kit with `make -C pintool PIN_ROOT=<path>`, and `scripts/run.sh` runs a workload natively and under the tool with these options.

B Output files

Each run writes files named `<Prefix>_<name>_<interval>_<pid>`, where the prefix is `Buffered` (splitter), `Sampled` (sampler) or `Spatial` (spatial).

- The summary file (`.csv`) and one file per bin (`_SubSample_<interval>_bin_<j>.csv`) have the columns `FunctionName`, `MemUsageObs` (footprint in bytes), `UniqueAddresses`, `CountObs` (references counted) and `SamplingInterval`, as in MemPrint.
- The timeline (`_timeline.csv`, with `-snapshot`) has one row per footprint per snapshot, with the columns `Time`, `SamplingInterval`, `Bin` (-1 for the main footprint, -2 for the union of an interval's bins), `MemUsageObs`, `UniqueAddresses`, `CountObs`, `FreedBytes`, `Singletons`, `Doubletons`, `Tripletons`, `Quadrupletons` (f_1 to f_4) and `Discovered` (how often an address entered the sample).
- The windowed file (`_windowed.csv`, with `-window`) has one row per snapshot, with the columns `Time`, `Watching` (1 inside a window), `Windows` (completed), `AllocatedBytes`, `Blocks`, `FreshBytes`, `FreshResident`, `ReusedBytes`, `ReusedResident`, `FreshEst`, `ReusedEst` and `OtherEst` (selected footprint $\times i$ in fresh blocks, in reused blocks and outside blocks), `FreshCovered` and `ReusedCovered` (bytes covered $\times i$), `Density` (of all reused blocks; 1 with bytes touched), `Estimate` ((4)), `Window`, `Period`, `ReusedWritten` (bytes of reused blocks on written pages), `TriggeredWindows` (windows opened by allocations), `WatchedRefs` (references watched so far), `RegionsResident` (resident bytes of the writable sections and the main stack, with `-pages_only`) and `PartialEdges` (touched pages inside blocks that border untouched ones).

The analysis package scores windowed runs against a splitter run in the same directory with `python -m memprint timeline windowed <workload>`, and `paper/make_figures.py` regenerates the figures of this report from the experiment outputs.

Table 13: Options of `memprint_trace`.

Option	Meaning
`-mode`	`splitter`, `sampler` or `spatial`.
`-i`	Sampler: sample 1 in i references. Spatial: select 1 in i words (start addresses with `-footprint starts`; 64-byte chunks in windows with `-footprint starts`).
`-s`	Sampler: number of bootstrap bins. Spatial: number of hash buckets.
`-r`	Sampler: replication factor, $\lambda = s/r$.
`-intervals`, `-bins`	Splitter: sampling intervals and bins per interval.
`-seed`	Seed of the random number generators and of the hash salt (default: the process id).
`-track_frees`	Remove freed ranges (malloc family and munmap) from all footprints, and exclude the allocator's own accesses.
`-footprint`	`bytes` (bytes touched) or `starts` (largest access per start address, as in MemPrint). Default: `bytes` with `-track_frees` or `-window`, else `starts`.
`-snapshot`	Write a timeline row set every N references.
`-stop`	Write the outputs after N references and detach.
`-buffer_pages`	Trace buffer pages per thread (default 1024, or 16 with snapshots or free tracking).
`-window`, `-period`	Spatial: check references only for W of every P references (requires `-snapshot`, and turns on free tracking).
`-window_alloc`	Windowed: an allocation of at least N bytes opens a window (default 1 MB; 0 turns this off).
`-dirty_every`	Windowed: clear soft-dirty bits every N references (default: every ten snapshots; 0: use residency for reused blocks).
`-pages_only`	Spatial with `-snapshot`: no windows; memory outside blocks from the resident pages of the writable image sections and the main stack.
`-window_switch`	Windowed: `flush`, `version`, or `auto` (default: flush until a window change retranslates more than `-version_traces` traces, by default 10000, then version).
`-edge_correction`	Windowed: count half a page less for each touched page inside a block that borders an untouched one (default on).
`-block_slack`	Windowed: measure heap blocks over their whole chunk, slack included (default off).
`-outdir`, `-name`	Output directory and trace name.